Discover Artificial Intelligence

RESEARCH Open Access

# An explainable XGBoost–MLP fusion framework for intrusion detection in 5G and beyond networks

Hani Al-Balasmeh[1*]

*Correspondence:
Hani Al-Balasmeh
h.albalasmeh@utb.edu.bh
[1]College of Engineering, University of Technology Bahrain (UTB), Salmabad, Kingdom of Bahrain

**Abstract**

Fifth-generation (5G) and beyond networks combine virtualization, network slicing, massive device connectivity, and multi-access edge computing, thereby increasing both service flexibility and cybersecurity exposure. This paper presents an explainable ensemble intrusion-detection framework that combines Extreme Gradient Boosting (XGBoost) and a compact multilayer perceptron (MLP) through validation-controlled probability fusion. The framework also integrates class-imbalance handling, feature selection, SHapley Additive exPlanations (SHAP), Local Interpretable Model-Agnostic Explanations (LIME), and explanation-aware security actions. Evaluation on the public 5G-NIDD benchmark used a stratified 70:15:15 training-validation-test split, common preprocessing for all models, and validation-only hyperparameter and fusion-weight selection. On the fixed held-out test split used in the supplied experiment log, the ensemble achieved 99.3% accuracy, 99.2% precision, 99.1% recall, 99.1% F1-score, 99.7% area under the receiver operating characteristic curve, and a 0.9% false-positive rate. The corresponding 95% Wilson interval for accuracy was 99.21-–99.38%, which describes sampling uncertainty on this test split rather than random-seed variability. SHAP identified packet rate, flow duration, SYN behavior, destination-port entropy, source bytes, and connection state as influential global indicators, while LIME supported alert-level validation. Detection required 22.6 ms per flow after warm-up, whereas generating both SHAP and LIME explanations increased end-to-end latency to 83.3 ms. These measurements motivate a two-path deployment strategy: continuous low-latency edge detection and selective explanation generation for high-risk, uncertain, or policy-sensitive alerts. A fixed-test statistical analysis using Wilson confidence intervals and Holm-adjusted accuracy comparisons confirmed that the reported improvement over every baseline was significant at adjusted $p < 0.001$ on the held-out sample; random-seed variability is stated separately as a limitation.

**Keywords** Fifth-generation networks, Beyond 5G, Network intrusion detection, Explainable artificial intelligence, Ensemble learning, XGBoost, Multilayer perceptron, SHAP, LIME, Edge computing

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

## 1 Introduction

Fifth-generation (5G) and beyond networks are reshaping modern communication infrastructures by enabling ultra-reliable low-latency communication, massive machine-type communication, enhanced mobile broadband, network slicing, software-defined networking (SDN), network function virtualization (NFV), cloud-native services, and multi-access edge computing (MEC). These capabilities provide the technical foundation for smart cities, autonomous transportation, industrial automation, intelligent healthcare, and large-scale Internet of Things (IoT) ecosystems. However, the same characteristics that make 5G and beyond networks flexible and service-oriented also increase their cybersecurity exposure. Unlike traditional static networks, next-generation networks are highly distributed, programmable, virtualized, and heterogeneous, which means that security threats may appear across the radio access network, core network, edge nodes, virtualized network functions, control-plane interfaces, user-plane traffic, and interconnected IoT devices [1, 2, 4].

The security challenge becomes more complex because 5G and beyond environments do not rely on a single network layer or a fixed perimeter. Network services are increasingly deployed through virtualized and software-controlled components, while network slicing allows multiple logical networks to operate over shared physical infrastructure. Although this design improves flexibility and service customization, it also introduces new attack surfaces related to slice isolation, signaling abuse, misconfigured virtual network functions, denial-of-service attacks, unauthorized access, data-plane manipulation, and attacks targeting edge resources. Recent security analyses have emphasized that 5G networks must be protected against threats affecting availability, confidentiality, integrity, authentication, orchestration, network management, and service continuity [2, 3, 5]. Therefore, intrusion detection in 5G and beyond networks must be adaptive, scalable, and capable of identifying both known and emerging attacks under highly dynamic traffic conditions.

Network intrusion detection systems (NIDSs) are among the most important defensive mechanisms for monitoring network traffic and identifying malicious behavior. Traditional signature-based systems are effective when attack patterns are already known, but they are usually limited in detecting unknown, polymorphic, or zero-day attacks. In contrast, machine learning and deep learning techniques can learn complex relationships from large-scale traffic data and can improve the detection of abnormal patterns that are difficult to capture using handcrafted rules. For this reason, artificial intelligence (AI)-based intrusion detection has become a major research direction in modern cybersecurity. However, the use of AI in 5G and beyond networks also introduces important concerns. Deep learning models may achieve high detection accuracy, but they often operate as black-box systems, making it difficult for security analysts to understand why a specific flow, session, or device behavior is classified as malicious [12, 42].

This lack of interpretability is not a minor issue in operational cybersecurity. In real network environments, an intrusion detection alert is not only a classification result; it is also a decision-support signal that may trigger blocking, isolation, forensic investigation, or policy adjustment. If the model cannot justify its prediction, network administrators may find it difficult to trust the alert, validate the cause of the detection, or distinguish between meaningful attack indicators and misleading correlations. This problem is especially critical in 5G and beyond networks because false positives may

disrupt latency-sensitive services, while false negatives may allow attacks to propagate through virtualized and edge-enabled infrastructure. Accordingly, an effective intrusion detection framework for next-generation networks should not only maximize accuracy but should also provide interpretable evidence that explains the reasoning behind each detection decision.

Explainable artificial intelligence (XAI) offers a promising direction for addressing this limitation. XAI methods aim to make AI-based decisions more transparent by identifying the features, patterns, or traffic characteristics that contribute most strongly to a prediction. Local explanation methods, such as Local Interpretable Model-Agnostic Explanations (LIME), can explain individual intrusion alerts by approximating the behavior of a complex model around a specific instance. Global explanation methods, such as Shapley Additive Explanations (SHAP), can identify the overall importance of traffic features across the detection model. In cybersecurity applications, these techniques can help analysts understand whether an alert is influenced by meaningful indicators such as abnormal packet rates, unusual flow duration, suspicious protocol behavior, repeated failed connections, or traffic patterns associated with denial-of-service, reconnaissance, exploitation, or botnet activity [11, 12, 42].

Despite the growing number of AI-based intrusion detection studies, several research gaps remain. First, many existing models focus mainly on detection accuracy while giving limited attention to interpretability and operational trust. Second, some studies apply explainability only as a post-processing visualization tool, without integrating it into a complete security decision framework. Third, many intrusion detection models are evaluated on general-purpose datasets without sufficiently discussing their relevance to distributed 5G and beyond network environments. Fourth, standalone machine learning or deep learning models may perform well under specific datasets but may be less robust when facing class imbalance, heterogeneous traffic behavior, and evolving attack patterns. Finally, there is still a need for a unified framework that combines ensemble-based detection with explainable decision support in a way that is technically suitable for future intelligent communication networks.

To address these limitations, this paper proposes a explainable ensemble framework for network intrusion detection in 5G and beyond networks. The proposed framework integrates data preprocessing, feature engineering, ensemble-based classification, and explainability-driven decision interpretation. The ensemble detection layer combines the strengths of machine learning and deep learning models to improve classification robustness, while the explainability layer uses SHAP and LIME to provide both global and local explanations. By combining detection performance with interpretable security reasoning, the framework aims to support accurate, transparent, and trustworthy intrusion detection for next-generation network environments.

Figure 1 provides a compact view of the study concept and clarifies the separation between the always-on detection path and the selectively invoked explanation path.

The main contributions of this paper are summarized as follows:

- An explainable probability-level ensemble is designed around complementary XGBoost and MLP branches rather than describing deep learning as a category separate from machine learning.

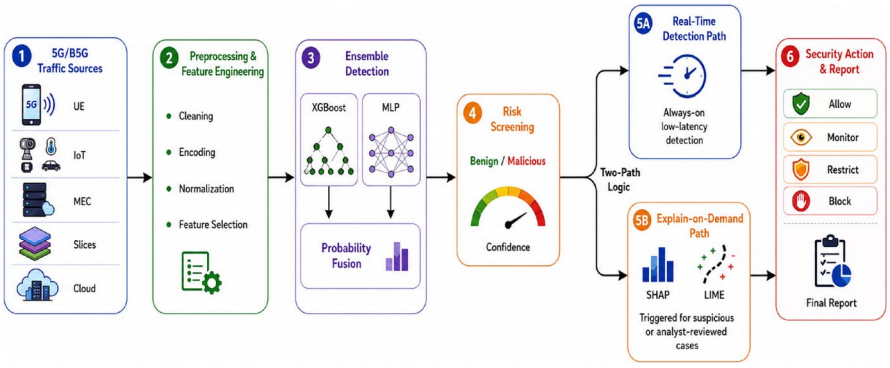



**Fig. 1** Conceptual overview of the explainable ensemble intrusion-detection workflow and its two-path deployment logic

- The fusion weight and all hyperparameters are selected only on the validation partition, while every baseline receives the same split and fitted preprocessing transformation to improve comparison fairness.
- A training-only feature-selection workflow combines identifier/leakage screening, mutual-information ranking, and recursive feature elimination, with the retained subset selected by validation macro-F1.
- SHAP and LIME outputs are connected to graded actions (allow, monitor, restrict, and block), and their fidelity, stability, and latency are evaluated rather than presented only as illustrative plots.
- The study reports component-level latency, analytical complexity, fixed-test statistical validation, error patterns, difficult attack classes, expanded ablations, and deployment limitations.

The remainder of this paper is organized as follows. Section 2 reviews existing studies on AI-based intrusion detection, explainable AI for cybersecurity, and security challenges in 5G and beyond networks. Section 3 presents the proposed explainable ensemble framework, including the system architecture, preprocessing phase, ensemble detection model, and explainability mechanism. Section 4 describes the experimental setup, datasets, baseline models, and evaluation metrics. Section 5 presents and discusses the experimental results, including detection performance, comparative analysis, ablation study, and explainability analysis. Finally, Sect. 6 concludes the paper and outlines future research directions.

## 2 Related work

Intrusion detection in 5G and beyond networks has received increasing attention because next-generation communication systems combine high-speed connectivity, virtualization, network slicing, edge computing, and massive IoT access within a highly dynamic security environment. Unlike conventional enterprise networks, 5G and beyond infrastructures are not limited to fixed endpoints or clearly defined network borders. They consist of distributed radio access components, core-network functions, software-defined control layers, virtualized services, edge nodes, and heterogeneous user devices. This architectural complexity makes attack detection more challenging and requires security solutions that can operate under changing traffic conditions, diverse attack behaviors, and strict latency requirements. In this context, artificial intelligence

(AI), machine learning (ML), deep learning (DL), and explainable artificial intelligence (XAI) have become important research directions for strengthening network intrusion detection.

### 2.1 Machine learning-based intrusion detection

Machine learning-based intrusion detection systems have been widely investigated as an alternative to conventional signature-based mechanisms. Signature-based systems are generally effective against known attacks, but their performance becomes limited when dealing with unknown, polymorphic, or zero-day threats. ML-based approaches attempt to overcome this limitation by learning statistical and behavioral patterns from network traffic data. Commonly used models include Decision Tree (DT), Random Forest (RF), Support Vector Machine (SVM), k-Nearest Neighbors (k-NN), Naive Bayes (NB), Logistic Regression (LR), Gradient Boosting, and Extreme Gradient Boosting (XGBoost). These models are often trained using flow-level features, protocol attributes, packet statistics, connection duration, source and destination behavior, and service-level indicators [8, 9, 15-17].

Several studies have shown that ML classifiers can achieve competitive detection performance when the training data are properly preprocessed and validated. However, model performance is strongly affected by feature quality, class imbalance, data leakage, and the representativeness of the dataset. In intrusion detection, this issue is particularly important because a model may appear to achieve high accuracy when the dataset contains redundant records or when training and testing samples share hidden dependencies. Therefore, recent work has emphasized the need for leakage-free validation, careful feature engineering, and realistic testing scenarios, especially when IDS models are evaluated for 5G and beyond environments [6, 8].

Although traditional ML models are generally easier to train and interpret than deep neural networks, they may struggle to capture complex temporal and spatial relationships in high-dimensional network traffic. For example, attacks such as distributed denial-of-service, slow-rate denial-of-service, reconnaissance, botnet communication, and multi-stage exploitation may involve sequential or correlated traffic patterns that are not fully represented by isolated flow statistics. As a result, ML-based IDS approaches remain useful, but they are often insufficient when used alone in complex, fast-changing 5G and beyond networks.

### 2.2 Deep learning-based intrusion detection

Deep learning has been increasingly adopted for intrusion detection because of its ability to learn complex representations from large-scale and high-dimensional network data. Models such as Multilayer Perceptron (MLP), Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), Autoencoder (AE), and hybrid CNN-LSTM architectures have been applied to classify benign and malicious traffic. These methods are particularly useful when the IDS must detect nonlinear relationships, hidden dependencies, or sequential attack behaviors in network flows [9, 10, 18-20].

In 5G and beyond environments, deep learning can support intrusion detection across distributed network functions, edge nodes, and IoT traffic streams. For instance, deep models can be used to detect denial-of-service attacks, port scanning, abnormal

signaling behavior, malicious traffic bursts, and suspicious service access patterns. The availability of 5G-oriented datasets, such as 5G-NIDD, has also encouraged researchers to evaluate IDS models using traffic collected from realistic 5G testbed environments. The 5G-NIDD dataset is particularly relevant because it contains labeled traffic extracted from a functional 5G test network and includes benign traffic as well as attack scenarios such as ICMP flood, UDP flood, SYN flood, HTTP flood, slow-rate denial-of-service, and port scanning [6, 7, 21-30].

Despite their strong classification ability, deep learning-based IDS models face several limitations. First, deep models usually require large amounts of representative training data, which may not always be available for newly emerging 5G and beyond attack scenarios. Second, they may be computationally expensive, especially when deployed in latency-sensitive edge environments. Third, deep models are often difficult to interpret because their decision-making process is distributed across many hidden layers and parameters. This black-box nature limits their practical adoption in security operation centers because analysts need to understand why an alert was generated before taking defensive action.

### 2.3 Ensemble and multi-model approaches for intrusion detection

Ensemble AI-based intrusion detection has emerged as a promising direction for improving detection robustness. Instead of relying on a single classifier, hybrid approaches combine multiple learning models or processing stages to benefit from their complementary strengths. For example, a hybrid IDS may combine feature selection with ensemble learning, use an autoencoder for feature extraction followed by a tree-based classifier, integrate CNN layers for spatial representation with LSTM layers for temporal learning, or fuse predictions from classical ML and DL models using voting or weighted averaging.

The main advantage of ensemble models is that they can reduce the weaknesses of individual classifiers. Tree-based models are effective for structured tabular traffic features and can provide useful feature-importance indicators. Deep neural networks can capture complex nonlinear and sequential patterns. Ensemble methods can improve generalization by combining multiple decision boundaries. In intrusion detection, this is valuable because network attacks differ in behavior, intensity, duration, and feature distribution. A model that performs well for denial-of-service attacks may not necessarily perform equally well for reconnaissance or low-rate attacks. Therefore, hybrid learning can provide a more balanced detection mechanism across multiple attack categories.

However, ensemble AI models also introduce new challenges. Combining several models can increase computational overhead, training complexity, and deployment cost. More importantly, ensemble models may become even less interpretable than standalone classifiers. If the final decision is produced through multiple processing layers or ensemble fusion, it becomes difficult to explain which traffic features or model components influenced the prediction. Therefore, hybrid IDS design should not focus only on improving accuracy. It should also include an explanation mechanism that helps security analysts understand the relationship between input traffic features and final detection decisions.

### 2.4 Explainable AI for intrusion detection

Explainable artificial intelligence has become an essential research direction for intrusion detection because cybersecurity decisions require transparency, accountability, and analyst trust. In practical environments, an IDS alert may lead to blocking a connection, isolating a device, changing an access policy, or starting a forensic investigation. If the IDS cannot explain its prediction, security teams may hesitate to act on the alert, especially when the predicted attack affects critical services or latency-sensitive network slices.

LIME and SHAP are among the most widely used post-hoc explanation methods for black-box models. LIME explains an individual prediction by approximating the complex model locally using a simpler interpretable model around the selected instance. This allows the analyst to identify which features contributed to a specific classification result [13]. SHAP, on the other hand, is based on Shapley values from cooperative game theory and assigns an importance value to each feature according to its contribution to the model output [14, 31-33]. While LIME is commonly used for local instance-level explanation, SHAP can support both local and global interpretation by identifying feature contributions across individual predictions and the entire dataset.

Recent cybersecurity studies have applied XAI methods to improve the interpretability of AI-based IDS models. For example, XAI can reveal whether an intrusion alert was mainly influenced by flow duration, packet rate, source bytes, destination bytes, protocol behavior, connection state, abnormal port usage, or other network features. Such explanations are useful because they help analysts verify whether the model is learning meaningful attack indicators rather than relying on accidental correlations or dataset-specific artifacts. This is especially important in 5G and beyond networks, where false positives can interrupt critical services and false negatives can allow attacks to spread through distributed edge and core components [11, 12, 42].

Nevertheless, existing XAI-based IDS research still has limitations. In some studies, explainability is applied only after the model has been trained and evaluated, without being integrated into the broader security response process. In other cases, explanations are presented as visual outputs without being connected to practical access control, alert prioritization, or decision support. Moreover, many XAI studies focus on general intrusion detection datasets and do not sufficiently address the special characteristics of 5G and beyond networks, such as network slicing, edge deployment, virtualized network functions, and heterogeneous IoT traffic.

### 2.5 Intrusion detection in 5G and beyond networks

The security requirements of 5G and beyond networks differ from those of traditional networks because the communication infrastructure is more distributed, programmable, and service-oriented. 5G systems rely on SDN, NFV, MEC, cloud-native network functions, and network slicing to support different services with different performance and security requirements. These technologies improve flexibility and scalability, but they also create new points of vulnerability. For example, attacks may target slice isolation, edge-hosted applications, virtualized network functions, control-plane signaling, user-plane forwarding, orchestration interfaces, and IoT access traffic [1, 2, 5, 34-36].

The integration of MEC is particularly important for intrusion detection. Edge computing can reduce detection latency by processing traffic closer to the data source,

but it also introduces security risks because edge nodes may be resource-constrained, physically distributed, and exposed to local attacks. Similarly, massive IoT connectivity increases the number of devices that may be compromised and used for botnets, distributed attacks, scanning campaigns, or unauthorized access attempts. Therefore, IDS models for 5G and beyond networks must be lightweight enough for near-real-time deployment, scalable enough for large traffic volumes, and accurate enough to detect diverse attacks across heterogeneous environments.

Another important issue is that different network slices may have different risk profiles. For example, a slice supporting autonomous vehicles or remote healthcare may require stricter detection thresholds than a slice supporting delay-tolerant IoT sensing. A practical IDS should therefore support context-aware detection and response rather than treating all network traffic equally. Explainable AI can support this requirement by helping security operators understand why a flow is suspicious and whether the decision is consistent with the service context and security policy.

Table 1 clarifies the state-of-the-art gap. Recent studies provide either strong detection performance, 5G-oriented benchmarking, or post-hoc explanation, but fewer studies connect XGBoost–MLP fusion, SHAP/LIME explanation, latency analysis, and explanation-aware security actions in a single operational framework. This motivates the proposed design.

Recent work further demonstrates why a broader comparison is needed. Transformer-based 5G-NIDD detection has reported strong multiclass performance but with increased computational time [43]. Graph-neural-network IDS research models relationships among hosts and flows and has improved multiclass attack separation on conventional benchmarks [44]. Federated-learning studies for 5G/B5G emphasize privacy and non-identically distributed slice data but also reveal convergence, communication, and heterogeneity challenges [45]. Explainable SDN and IoT IDS studies have combined feature optimization or deep belief networks with interpretable outputs [46, 47], while ExAI5G couples a Transformer detector with logic-based explanations for 5G traffic [48]. These approaches are valuable comparators, but they do not evaluate the same combination of XGBoost–MLP probability fusion, SHAP/LIME agreement, graded response mapping, and two-path edge deployment examined here.

### 2.6 Research gap and motivation

Based on the reviewed literature, several gaps can be identified. First, many ML- and DL-based IDS studies focus primarily on improving classification performance, while interpretability is treated as a secondary concern. This limits the practical usability of these models in real security operations, where analysts need clear explanations before taking action. Second, existing XAI-based IDS approaches often provide local or global explanations but do not integrate these explanations into a unified detection and security decision framework. Third, many proposed IDS models are evaluated on general-purpose datasets without clearly demonstrating their suitability for 5G and beyond network characteristics. Fourth, standalone classifiers may not provide sufficient robustness across diverse attack categories, while ensemble models can improve detection capability but may increase black-box complexity. Finally, there remains a need for an IDS framework that jointly addresses detection accuracy, model interpretability, and operational decision support in edge-enabled and virtualized next-generation networks.

**Table 1** Summary of representative state-of-the-art IDS and XAI studies relevant to the proposed framework

| Study | Main approach | Dataset/context | Reported focus | XAI / decision support | Remaining limitation addressed in this paper |
|---|---|---|---|---|---|
| Samarakoon et al. [6] | ML benchmarking | 5G-NIDD | 5G testbed IDS evaluation | Limited | Provides a relevant dataset but focuses mainly on conventional detection performance |
| Gaspar et al. [11] | MLP with LIME/SHAP | IDS benchmark data | XAI applicability | LIME and SHAP | Explains a single neural model but does not integrate probability fusion with operational response |
| Bouke et al. [8] | Empirical ML evaluation | IDS benchmark data | Validation and model comparison | Limited | Highlights the need for careful evaluation but does not focus on 5G operational decision support |
| Javid et al. [10] | Lightweight DL IDS | Network/IoT security context | Efficient detection | Limited | Emphasizes efficiency but provides limited alert-level reasoning |
| Nugraha et al. [40] | XAI-enabled IDS framework | Network security datasets | Robust and explainable IDS | XAI-based interpretation | Provides explainability but is not centered on XGBoost–MLP fusion for 5G-NIDD and response actions |
| Ilias et al. [41] | CNN and mixture-of-experts | 5G-NIDD and O-RAN dataset | High-performance 5G IDS | Limited | Shows strong 5G detection performance, but explainability and operational response are not central |
| **This study** | **XGBoost–MLP fusion with SHAP and LIME** | **5G-NIDD** | **Detection, explanation, latency, and response** | **Global/local XAI with action mapping** | **Jointly addresses accuracy, interpretability, reproducibility, and decision support.** |

ML, machine learning; DL, deep learning; IDS/NIDS, intrusion/network intrusion detection system; XAI, explainable artificial intelligence; O-RAN, Open Radio Access Network

Motivated by these gaps, this paper proposes an explainable ensemble intrusion-detection framework for 5G and beyond networks. The proposed framework integrates ensemble AI classification with SHAP- and LIME-based explanation mechanisms to support both accurate detection and transparent security reasoning. Unlike conventional IDS approaches that only report whether traffic is benign or malicious, the proposed framework aims to provide interpretable evidence behind each decision, enabling security analysts to validate alerts, understand feature-level attack indicators, and support response actions in complex next-generation network environments.

### 2.7 Research objectives, questions, and operational scenario

In response to the need for clearer research design, this study is guided by four concrete objectives: (i) to design a XGBoost–MLP NIDS that can detect benign and malicious 5G traffic with high reliability, (ii) to evaluate whether weighted fusion improves detection performance compared with individual ML and DL baselines, (iii) to determine whether SHAP and LIME provide actionable explanations for global model behavior and individual alerts, and (iv) to assess whether the framework can support practical decision-making under edge-oriented latency constraints. The main research questions are therefore: RQ1: Can a XGBoost–MLP model improve intrusion detection performance on 5G-NIDD compared with standalone baseline models? RQ2: Can SHAP and LIME explain the most important traffic indicators behind both global and local detection decisions? RQ3: Can explanation evidence be connected to practical security actions such as allow, monitor, restrict, or block? RQ4: Is the latency profile compatible with a two-level 5G deployment scenario in which rapid detection is performed at the edge and detailed explanations are generated selectively for high-risk alerts? The intended operational users are telecom operators, mobile network security teams, enterprise security operation centers, MEC administrators, and researchers developing trustworthy NIDS solutions for 5G/B5G and future 6G environments.

## 3 Proposed explainable ensemble framework

This section presents the proposed explainable ensemble framework for network intrusion detection in 5G and beyond networks. The framework is designed to address three key requirements of modern network security: accurate attack detection, interpretable decision-making, and actionable security response. In contrast to conventional intrusion detection systems that mainly provide a classification result, the proposed framework generates a complete security output that includes the predicted class, attack probability, explanation evidence, and recommended response action.

The proposed framework is suitable for 5G and beyond environments because these networks are highly distributed, heterogeneous, software-defined, and edge-enabled. Traffic may originate from user equipment, IoT devices, multi-access edge computing (MEC) nodes, virtualized network functions, network slices, and cloud/core services. Such diversity increases the attack surface and makes static or signature-based intrusion detection insufficient for identifying evolving and complex threats. Therefore, the proposed framework integrates ensemble-based detection with explainable AI techniques to support both high detection performance and transparent security reasoning.

Figure 2 illustrates the overall architecture of the proposed framework. The architecture consists of seven main layers: the 5G/B5G network environment, traffic collection

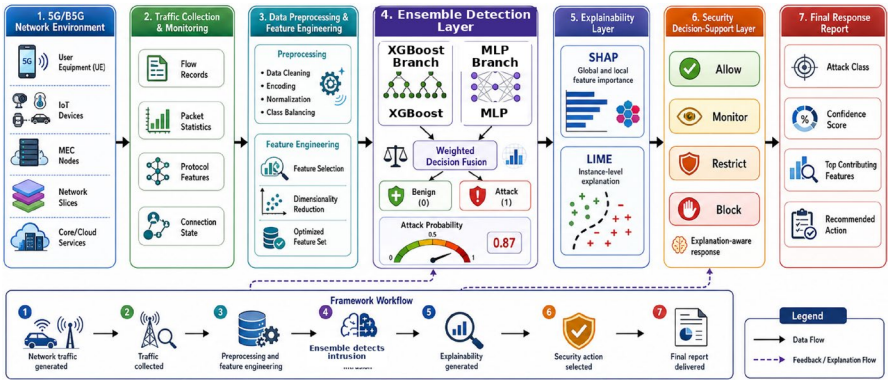



**Fig. 2** Overall architecture of the proposed explainable ensemble framework for network intrusion detection in 5G and beyond networks

and monitoring, data preprocessing and feature engineering, ensemble-based detection, explainability, security decision support, and final response reporting.

### 3.1 System architecture

As shown in Fig. 2, the proposed framework begins with the 5G/B5G network environment layer. This layer represents heterogeneous traffic sources, including user equipment, IoT devices, MEC nodes, network slices, and cloud/core services. These components generate different types of traffic flows, including normal communication patterns and malicious activities such as denial-of-service attacks, scanning, exploitation attempts, abnormal signaling, unauthorized access, and compromised IoT traffic.

The second layer is the traffic collection and monitoring layer. This layer collects network traffic information from monitoring points located at the edge, core, or cloud level. The collected data may include flow records, packet statistics, protocol features, connection states, and system or log-based information. These records represent the raw input used by the learning model to distinguish benign and malicious behavior.

The third layer performs data preprocessing and feature engineering. This stage is necessary because raw network traffic often contains missing values, duplicate records, invalid entries, categorical variables, noisy attributes, and imbalanced class distributions. Data preprocessing improves the quality of the input dataset, while feature engineering identifies the most relevant attributes for intrusion detection. This step reduces computational complexity and improves the interpretability of the detection model.

The fourth layer is the ensemble detection layer. It combines two complementary estimators trained on the same selected representation. The XGBoost branch captures structured and threshold-based feature interactions, while the MLP branch captures dense nonlinear relationships. Their calibrated class-probability outputs are combined through validation-controlled weighted fusion to generate the final prediction.

The fifth layer is the explainability layer. It uses SHAP and LIME to provide global and local explanations for the intrusion detection decisions. SHAP identifies the most important features affecting the model output across the dataset, while LIME explains why a specific traffic instance is classified as benign or malicious. This layer improves the transparency of the detection process and supports analyst trust in AI-generated alerts.

The sixth layer is the security decision-support layer. This layer converts the prediction probability and explanation evidence into an actionable security response. The possible

responses include allowing the traffic, monitoring suspicious behavior, restricting access, or blocking the traffic. This decision is not based only on the predicted class but also on the attack probability and explanation report.

The final layer produces the response report. This report includes the predicted attack class, confidence score, top contributing features, explanation summary, and recommended action. The report can be used by security operation centers, edge security controllers, or automated policy enforcement systems to support rapid and interpretable cybersecurity response.

### 3.2 Data representation and problem formulation

Let the monitored network traffic dataset be represented as:

$$D = \{(x_i, y_i)\}_{i=1}^{N}, \tag{1}$$

where $D$ is the dataset, $x_i$ is the feature vector of the $i$-th traffic instance, $y_i$ is the corresponding label, and $N$ is the total number of traffic instances.

Each traffic instance is represented by a set of features:

$$x_i = \{f_1, f_2, \ldots, f_n\}, \tag{2}$$

where $f_1, f_2, \ldots, f_n$ denote the extracted traffic features, such as flow duration, packet count, byte count, protocol type, connection state, packet rate, source and destination ports, and other statistical or behavioral indicators.

For binary intrusion detection, the label space is defined as:

$$y_i = \begin{cases} 0, & \text{benign traffic,} \\ 1, & \text{malicious traffic.} \end{cases} \tag{3}$$

For multi-class intrusion detection, the label space is defined as:

$$y_i \in \{0, 1, 2, \ldots, K\}, \tag{4}$$

where 0 represents benign traffic and 1 to $K$ represent different attack categories.

The objective of the proposed framework is to learn a detection function:

$$\mathcal{F} : X \rightarrow \mathcal{Y}, \tag{5}$$

where $X$ is the input feature space and $\mathcal{Y}$ is the class label space. In addition, the framework generates an explanation function:

$$\mathcal{G} : (x_i, \hat{y}_i) \rightarrow E_i, \tag{6}$$

where $\hat{y}_i$ is the predicted class and $E_i$ is the explanation report showing the most influential features behind the decision.

### 3.3 Data preprocessing and feature engineering

The preprocessing pipeline was fitted on the training partition only and then frozen before transforming the validation and test partitions. Duplicate records and invalid rows were removed, missing numerical values were imputed using training-set medians, and categorical values were encoded using categories learned from the training data. Source/destination identifiers, timestamps that could reveal the capture sequence, and

any field derived directly from the class label were excluded before selection to reduce identifier memorization and target leakage.

For a raw dataset $D_{raw}$, the cleaned representation is

$$D_{clean} = \mathcal{P}(D_{raw}), \tag{7}$$

where $\mathcal{P}(\cdot)$ denotes the training-fitted cleaning, encoding, and scaling pipeline. Numerical variables supplied to the MLP were min–max normalized as

$$x'_{ij} = \frac{x_{ij} - \min_{train}(x_j)}{\max_{train}(x_j) - \min_{train}(x_j)}, \tag{8}$$

with extrema computed only from the training partition. The tree branch received the same encoded feature columns; applying an identical frozen transformation avoids giving a baseline or branch access to different information.

Class imbalance was handled with inverse-frequency class weights,

$$w_k = \frac{N}{KN_k}, \tag{9}$$

where $N$ is the number of training samples, $K$ is the number of classes, and $N_k$ is the training count for class $k$.

Feature selection followed four reproducible stages. First, constant, near-constant, duplicate, identifier, and leakage-prone variables were removed. Second, mutual information was calculated on the training data,

$$I(f_j; Y) = \sum_{f_j} \sum_{y} p(f_j, y) \log\left(\frac{p(f_j, y)}{p(f_j)p(y)}\right). \tag{10}$$

Third, the ranked candidates were refined using recursive feature elimination (RFE) with an XGBoost estimator and macro-F1 on the validation partition. Fourth, the smallest subset whose validation macro-F1 was within 0.1 percentage point of the best observed subset was retained. The selected columns and their order were then frozen and used by every baseline, both ensemble branches, and every ablation except the explicit "without feature selection" variant. This procedure prevents feature-selection information from leaking from the test set and makes the selection criterion operationally clear.

### 3.4 Ensemble intrusion detection model

The proposed detector combines an XGBoost branch and an MLP branch. XGBoost is effective for structured traffic variables and threshold-based interactions, whereas the MLP learns dense nonlinear combinations of the same normalized features. Fusing their calibrated probability outputs reduces dependence on either estimator and improves robustness across heterogeneous traffic and attack categories.

For a given traffic instance $x_i$, the XGBoost branch produces the class probability:

$$P_{ML}(y = c|x_i) = M_{ML}(x_i), \tag{11}$$

where $M_{ML}$ is the machine learning classifier and $c$ is a possible class label.

The MLP branch produces:

$$P_{DL}(y = c|x_i) = M_{DL}(x_i), \tag{12}$$

where $M_{DL}$ is the deep learning classifier.

The final ensemble probability is calculated using weighted decision fusion:

$$P_h(y = c|x_i) = \alpha P_{ML}(y = c|x_i) + (1 - \alpha)P_{DL}(y = c|x_i), \tag{13}$$

where $\alpha \in [0, 1]$ controls the contribution of each branch.

The final predicted class is obtained as:

$$\hat{y}_i = \arg\max_{c \in \mathcal{Y}} P_h(y = c|x_i). \tag{14}$$

This fusion strategy allows the framework to combine the reliability of machine learning with the representational power of deep learning. The fusion weight $\alpha$ can be selected empirically using the validation set.

For reproducibility, the fusion parameter $\alpha$ was selected using validation-set optimization rather than manual adjustment. A grid search was performed over $\alpha \in \{0.0, 0.1, 0.2, \ldots, 1.0\}$, and the value that maximized validation macro-F1 while maintaining a low false positive rate was retained. The final reported experiments used $\alpha = 0.6$, giving slightly greater weight to the ML branch while preserving the nonlinear contribution of the DL branch. This setting was fixed before testing and was not retuned on the test set.

### 3.5 Training objective

The model is trained by minimizing the difference between the predicted probability distribution and the true class labels. For multi-class intrusion detection, weighted categorical cross-entropy is used:

$$\mathcal{L}_{WCE} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{K} w_c y_{ic} \log(\hat{p}_{ic}), \tag{15}$$

where $w_c$ is the class weight, $y_{ic}$ is the true class indicator, and $\hat{p}_{ic}$ is the predicted probability for class $c$.

To improve generalization and reduce overfitting, the complete optimization objective is defined as:

$$\theta^* = \arg\min_{\theta} \left( \mathcal{L}_{WCE} + \lambda \Omega(\theta) \right), \tag{16}$$

where $\theta$ represents the trainable model parameters, $\Omega(\theta)$ is the regularization term, and $\lambda$ controls the regularization strength.

### 3.6 Explainability mechanism

The explainability mechanism is included to make the ensemble detection model transparent and useful for security analysts. Since black-box predictions are difficult to trust in operational cybersecurity environments, the proposed framework integrates SHAP and LIME to explain why traffic is classified as benign or malicious.

SHAP provides feature-attribution values based on Shapley values. The SHAP explanation model is expressed as:

$$g(z') = \phi_0 + \sum_{j=1}^{M} \phi_j z'_j, \tag{17}$$

where $\phi_0$ is the base value, $\phi_j$ is the Shapley value of feature $j$, and $z'_j$ indicates whether feature $j$ is present in the simplified input representation.

The Shapley value for feature $j$ is computed as:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \left[ v(S \cup \{j\}) - v(S) \right], \tag{18}$$

where $S$ is a subset of features and $v(S)$ is the model output based on that subset.

LIME explains an individual prediction by approximating the complex model with a simpler local surrogate model around the selected instance. The LIME objective is defined as:

$$\xi(x_i) = \arg\min_{g \in G} \mathcal{L}(f, g, \pi_{x_i}) + \Omega(g), \tag{19}$$

where $f$ is the original ensemble model, $g$ is the interpretable surrogate model, $\pi_{x_i}$ defines the locality around instance $x_i$, and $\Omega(g)$ penalizes explanation complexity.

The combined use of SHAP and LIME provides two complementary levels of interpretability. SHAP supports global understanding of feature importance, while LIME provides local explanations for individual intrusion alerts. This combination helps analysts verify whether a prediction is based on meaningful traffic behavior rather than dataset-specific artifacts.

### 3.7 Explanation-aware security decision support

The security decision-support layer converts the detection result and explanation evidence into an operational response. Let $P_{attack}(x_i)$ denote the attack probability assigned to instance $x_i$. The response action $A_i$ is determined as:

$$A_i = \begin{cases} Allow, & P_{attack}(x_i) < \tau_1, \\ Monitor, & \tau_1 \le P_{attack}(x_i) < \tau_2, \\ Restrict, & \tau_2 \le P_{attack}(x_i) < \tau_3, \\ Block, & P_{attack}(x_i) \ge \tau_3, \end{cases} \tag{20}$$

where $\tau_1$, $\tau_2$, and $\tau_3$ are configurable decision thresholds.

The final response report is represented as:

$$R_i = \{\hat{y}_i, P_{attack}(x_i), E_i, A_i\}, \tag{21}$$

where $\hat{y}_i$ is the predicted class, $P_{attack}(x_i)$ is the attack probability, $E_i$ is the explanation report, and $A_i$ is the recommended response action.

This output allows the proposed framework to move beyond simple classification. It provides an interpretable and actionable result that can support alert prioritization, traffic monitoring, access restriction, automated blocking, and forensic analysis.

### 3.8 Algorithmic description

Algorithm 1 separates training-time model selection from test-time inference and explanation. This separation is important because preprocessing, feature selection, hyperparameter optimization, and fusion-weight selection must not use the held-out test labels.

**Require:** Labeled flow records $D$, candidate branches $M_X$ and $M_N$, fusion grid $A$, explanation policy $\pi$
**Ensure:** Frozen pipeline, predicted class $\hat{y}$, confidence $q$, explanation $E$, and action $a$
Stratify $D$ into training, validation, and held-out test partitions
Fit cleaning, encoding, imputation, and scaling on training data only
Remove identifiers/leakage fields and rank remaining features by mutual information
Run RFE on training data and select the smallest near-optimal subset using validation macro-F1
Apply the frozen transformation and selected feature order to all partitions
Tune XGBoost and MLP hyperparameters on the validation partition under matched search budgets
**for** each $\alpha \in A$ **do**
    Fuse validation probabilities $P = \alpha P_X + (1-\alpha)P_N$
    Record macro-F1 and false-positive rate
**end for**
Freeze the branch parameters and the best validation-controlled $\alpha$
Evaluate all baselines, branches, ensemble, and ablations once on the held-out test set
**for** each incoming flow $x$ **do**
    Transform $x$ using the frozen preprocessing and feature map
    Compute $P_X(\cdot|x)$ and $P_N(\cdot|x)$ in parallel
    Compute $P(\cdot|x) = \alpha P_X + (1-\alpha)P_N$
    Set $\hat{y} = \arg\max_c P(y=c|x)$ and $q = \max_c P(y=c|x)$
    **if** $\pi(\hat{y}, q, x)$ requests explanation **then**
        Generate SHAP global/local attribution and LIME local attribution
        Check explanation agreement and construct evidence report $E$
    **else**
        Set $E$ to a deferred-explanation token
    **end if**
    Map $(\hat{y}, q, E)$ to allow, monitor, restrict, or block
**end for**
**return** frozen artifacts and all evaluation outputs

**Algorithm 1** Explainable Ensemble Training, Evaluation, and Deployment

Let $N$ be the number of training flows, $d$ the retained feature dimension, $T$ the number of XGBoost trees, $h$ their maximum depth, and $E$ the number of MLP epochs. The approximate training costs are $O(TNd \log N)$ for histogram/tree boosting and $O(EN \sum_l n_{l-1} n_l)$ for the dense network. Per-flow detection is approximately $O(Th + \sum_l n_{l-1} n_l)$, excluding preprocessing. Exact SHAP cost depends on the explainer and tree structure, while LIME adds $O(md)$ perturbation generation plus surrogate fitting for $m$ local samples. Consequently, explanation is not placed on the mandatory path for every flow.

The computational complexity of the principal framework components is summarized in Table 2.

### 3.9 Deployment considerations

The proposed framework can be deployed in centralized, edge-based, or hybrid security architectures. In a centralized deployment, traffic features are collected from multiple network points and processed by a central security analytics platform. This approach provides broad visibility, but it may introduce additional communication overhead. In an edge-based deployment, lightweight detection can be performed close to MEC nodes to support low-latency response for time-sensitive services. In a hybrid deployment,

**Table 2** Computational-complexity summary of the proposed framework

| Component | Training/construction complexity | Per-flow inference/explanation complexity |
|---|---|---|
| XGBoost branch | $O(TNd \log N)$ | $O(Th)$ |
| MLP branch | $O(EN \sum_l n_{l-1} n_l)$ | $O(\sum_l n_{l-1} n_l)$ |
| Probability fusion | No additional model training | $O(C)$ for $C$ output classes |
| TreeSHAP | Built from the trained tree ensemble | Approximately $O(TLD^2)$ for $T$ trees, maximum leaves $L$, and depth $D$ |
| LIME | Local surrogate fitting for $m$ perturbations | $O(md)$ for perturbation generation, plus surrogate fitting |
| Decision support | Rule definition only | $O(C + k)$ for class probabilities and top-$k$ evidence |

$N$, training flows; $d$, retained features; $T$, trees; $h$, tree depth; $E$, epochs; $n_l$, units in layer $l$; $C$, classes; $m$, LIME perturbations. The expressions describe asymptotic scaling and complement the measured latency reported later

real-time detection can be performed at the edge, while detailed explanation and policy analysis can be performed at the central security layer.

This deployment flexibility makes the framework suitable for 5G and beyond networks. Edge-level detection supports rapid response, while centralized explanation supports deeper security analysis, alert validation, model refinement, and policy updates. Therefore, the proposed framework can support both real-time intrusion detection and explanation-aware cybersecurity management.

### 3.10 Methodology summary

The proposed methodology integrates traffic monitoring, preprocessing, feature engineering, ensemble AI detection, explainable interpretation, and decision support into a unified intrusion detection framework. The framework first transforms raw 5G/B5G traffic into a clean and optimized feature representation. It then applies an ensemble detector that combines XGBoost and MLP probabilities through weighted fusion. After prediction, SHAP and LIME are used to explain model decisions at global and local levels. Finally, the decision-support layer converts the predicted class, confidence score, and explanation evidence into an actionable response.

By combining detection accuracy with explanation-aware security reasoning, the proposed framework addresses the limitations of black-box IDS models and provides a practical foundation for trustworthy intrusion detection in complex 5G and beyond network environments.

## 4 Experimental setup

This section describes the experimental design used to evaluate the proposed explainable ensemble intrusion-detection framework. The evaluation is structured to examine three main aspects: detection performance, model robustness, and interpretability. Detection performance is measured using standard classification metrics, including accuracy, precision, recall, F1-score, false positive rate, and area under the receiver operating characteristic curve. Model robustness is evaluated through comparison with baseline machine learning and deep learning models. Interpretability is assessed using SHAP- and LIME-based explanation outputs to determine whether the proposed framework provides meaningful and analyst-supportive explanations for intrusion detection decisions.

The experimental setup is designed to reflect the requirements of 5G and beyond network security. Since next-generation networks are characterized by heterogeneous traffic

sources, distributed edge services, and dynamic attack behaviors, the evaluation focuses not only on classification accuracy but also on the ability of the framework to explain why a traffic instance is classified as benign or malicious. This is particularly important because a high-performing intrusion detection model may still be unsuitable for operational deployment if its decisions are not interpretable or trusted by security analysts.

### 4.1 Dataset description

The experimental evaluation used the public 5G-NIDD dataset because it was generated from a functional 5G testbed and is therefore more suitable for 5G/B5G intrusion detection than older enterprise-network benchmarks. The dataset was collected from traffic extracted through two base stations attached to the 5G Test Network at the University of Oulu, Finland, where attacker nodes targeted a server deployed in a MEC environment. The attack scenarios include denial-of-service and port-scanning traffic, namely ICMP Flood, UDP Flood, SYN Flood, HTTP Flood, Slow-rate DoS, SYN Scan, TCP Connect Scan, and UDP Scan [6, 7].

To improve clarity and reproducibility, the experimental setting was defined as a multi-class classification task using the major benign and malicious traffic categories available in 5G-NIDD. The complete published 5G-NIDD distribution contains 477,737 benign flows, 140,812 HTTP Flood flows, 1,155 ICMP Flood flows, 9,721 SYN Flood flows, 20,043 SYN Scan flows, 73,124 Slow-rate DoS flows, 20,052 TCP Connect Scan flows, 457,340 UDP Flood flows, and 15,906 UDP Scan flows. No synthetic data augmentation was applied. Class imbalance was handled through stratified splitting, class weighting, and validation-based model selection rather than by creating artificial traffic records.

The dataset used in the experiment is represented as:

$$D = \{(x_i, y_i)\}_{i=1}^{N}, \tag{22}$$

where $D$ denotes the complete dataset, $x_i$ is the traffic feature vector of the $i$-th sample, $y_i$ is the corresponding class label, and $N$ is the total number of samples.

The main characteristics of the 5G-NIDD dataset used in this study are summarized inTable 3.

### 4.2 Data preparation

Before model training, the dataset is prepared using a structured preprocessing procedure. First, duplicate records are removed to prevent repeated samples from artificially

**Table 3** Detailed description of the dataset used for experimental evaluation

| Item | Description |
|---|---|
| Dataset | 5G-NIDD public benchmark generated from a functional 5G testbed |
| Network context | 5G Test Network with traffic involving benign users, attacker nodes, and a MEC server |
| Total published flows | 1,215,890 flows across benign and attack classes |
| Main classes | Benign, ICMP Flood, UDP Flood, SYN Flood, HTTP Flood, Slow-rate DoS, and port-scan categories |
| Features | Flow-level, packet-level, protocol-based, statistical, and connection-state indicators |
| Augmentation | No synthetic data generation; imbalance handled through class weighting and stratified splitting |
| Task | Multi-class intrusion detection with binary benign/malicious interpretation available for operational response |

MEC, multi-access edge computing; 5G-NIDD, 5G Network Intrusion Detection Dataset; DoS, denial of service

increasing the classification performance. Second, missing and invalid values are handled to ensure that all feature vectors are complete and suitable for model training. Third, categorical features are encoded into numerical form using label encoding or one-hot encoding, depending on the feature type. Fourth, numerical features are normalized to reduce scale differences among variables.

The cleaned dataset is represented as:

$$D_{clean} = \mathcal{P}(D), \tag{23}$$

where $\mathcal{P}(\cdot)$ denotes the preprocessing function.

The dataset is then divided into training, validation, and testing subsets:

$$D_{clean} = D_{train} \cup D_{val} \cup D_{test}, \tag{24}$$

where $D_{train}$ is used to train the models, $D_{val}$ is used for hyperparameter tuning and fusion-weight selection, and $D_{test}$ is used for final performance evaluation. To ensure fair evaluation, the test set is kept unseen during the training and validation stages.

In this study, the dataset is divided using a stratified splitting strategy to preserve the distribution of benign and attack classes across the training, validation, and testing subsets. This is important because intrusion detection datasets often contain imbalanced class distributions. Without stratification, minority attack classes may be underrepresented in the validation or testing stage, resulting in unreliable performance estimates.

The split was implemented as 70% training, 15% validation, and 15% testing using stratified sampling and a fixed random seed of 42. The same partitions were used for all baseline models and the proposed framework. Duplicate removal, missing-value processing, encoding, normalization, feature selection, class weighting, and model fitting were performed inside the training/validation workflow to reduce leakage risk. The test set was used only once for final reporting. The training, validation, and testing split used in the experiments is summarized in Table 4.

### 4.3 Baseline models and fair-comparison protocol

The baselines represent linear, tree, ensemble, margin-based, feed-forward, recurrent, and convolutional-recurrent IDS families. Every model used the same stratified split, training-fitted preprocessing, selected feature order, class labels, validation objective, and held-out test set. Hyperparameters were optimized only on the validation partition. Search budgets were matched as closely as possible within each model family, and no model was tuned on the test set. The rationale for the selected baseline models and their validation-controlled search spaces is presented in Table 5.

The same metric definitions and post-processing rules were applied to all models. Training time and inference time are model- and hardware-dependent; the accompanying profiling script records wall-clock time, peak process memory, serialized model size, and available CPU/GPU utilization during a complete rerun. Because these values were not retained in the original experiment log, they are not fabricated in the revision.

**Table 4** Data splitting strategy used for model training, validation, and testing

| Subset | Suggested ratio | Purpose |
|---|---|---|
| Training set | 70% | Model learning and parameter optimization |
| Validation set | 15% | Hyperparameter tuning and fusion-weight selection |
| Testing set | 15% | Final evaluation on unseen data |

**Table 5** Baseline rationale and validation search space

| Model | Reason for inclusion | Validation-controlled settings |
|---|---|---|
| Logistic Regression | Linear probability baseline | Regularization *C* and class weighting |
| Decision Tree | Interpretable nonlinear baseline | Maximum depth, minimum leaf size, split criterion |
| Random Forest | Bagged-tree ensemble | Number of trees, maximum depth, sampled features |
| SVM | Margin-based nonlinear baseline | Kernel, *C*, and kernel coefficient |
| XGBoost | Strong structured-data branch/baseline | Estimators, depth, learning rate, row/column subsampling |
| MLP | Dense nonlinear branch/baseline | Layer width, dropout, learning rate, batch size, patience |
| LSTM | Sequence-oriented neural baseline | Hidden units, dropout, learning rate, sequence window |
| CNN–LSTM | Local-plus-sequential neural baseline | Filter count, kernel width, recurrent units, dropout |
| Proposed ensemble | Complementary probability fusion | All branch settings above plus fusion weight $\alpha$ |

### 4.4 Model configuration

The proposed ensemble model consists of two main detection branches. The first branch is a machine learning classifier trained on the selected feature subset. The second branch is a deep learning classifier trained using the same optimized feature representation. The probability outputs of both branches are fused using a weighted averaging mechanism.

The fusion weight $\alpha$ is selected using the validation set. Different values of $\alpha$ are tested to identify the best balance between the XGBoost and MLP branches. The final fused probability is computed as:

$$P_h(y = c|x_i) = \alpha P_{ML}(y = c|x_i) + (1 - \alpha)P_{DL}(y = c|x_i). \tag{25}$$

The value of $\alpha$ controls the relative contribution of each branch. When $\alpha$ is closer to 1, the ensemble gives more importance to XGBoost. When $\alpha$ is closer to 0, it gives more importance to the MLP. This flexibility allows the proposed model to adapt to the characteristics of the dataset.

The final reported configuration used XGBoost as the ML branch and a compact MLP as the DL branch. The MLP contained three dense layers with 128, 64, and 32 units, ReLU activation, dropout of 0.20 after the first two layers, and a softmax output layer. The model was trained with Adam optimizer, learning rate $10^{-3}$, batch size 256, early stopping with patience of 10 epochs, and a maximum of 100 epochs. Weighted categorical cross-entropy was used to reduce the effect of class imbalance.

A representative configuration of the proposed model is shown in Table 6.

### 4.5 Evaluation metrics

The proposed framework is evaluated using standard classification metrics commonly used in intrusion detection research. These metrics are calculated using true positives, true negatives, false positives, and false negatives. In cybersecurity applications, accuracy alone is not sufficient because a model may achieve high accuracy while failing to detect minority attack classes. Therefore, precision, recall, F1-score, false positive rate, and detection rate are also used.

Accuracy measures the overall proportion of correctly classified samples:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \tag{26}$$

**Table 6** Final configuration of the proposed explainable ensemble

| Component | Configuration |
|---|---|
| Input data | Selected, encoded, and normalized 5G-NIDD traffic features |
| XGBoost branch | Validation-tuned estimator count, depth, learning rate, and row/column subsampling; exact final values must be verified from the author rerun configuration |
| MLP branch | MLP with 128–64–32 hidden units, ReLU activation, dropout 0.20, and softmax output |
| Fusion strategy | Validation-optimized weighted decision fusion with $\alpha = 0.6$ |
| Loss function | Weighted categorical cross-entropy for multi-class classification |
| Optimization | Adam optimizer; learning rate $10^{-3}$; batch size 256; early stopping; maximum 100 epochs |
| Explainability methods | SHAP for global/local attribution and LIME for instance-level alert explanation |
| Security output | Predicted class, attack probability, top contributing features, and recommended action |

MLP, multilayer perceptron; SHAP, SHapley Additive exPlanations; LIME, Local Interpretable Model-Agnostic Explanations

Precision measures the proportion of predicted attacks that are truly malicious:

$$Precision = \frac{TP}{TP + FP}. \tag{27}$$

Recall, also known as detection rate, measures the proportion of actual attacks correctly detected:

$$Recall = \frac{TP}{TP + FN}. \tag{28}$$

The F1-score provides a balanced measure between precision and recall:

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}. \tag{29}$$

The false positive rate measures the proportion of benign samples incorrectly classified as attacks:

$$FPR = \frac{FP}{FP + TN}. \tag{30}$$

For multi-class classification, macro-averaged and weighted-averaged metrics are used. Macro-averaging gives equal importance to all classes, while weighted-averaging considers the number of samples in each class. This is important because intrusion detection datasets often contain imbalanced class distributions.

The area under the receiver operating characteristic curve is also used to evaluate the separability between benign and malicious traffic:

$$AUC = \int_0^1 TPR(FPR)\, d(FPR). \tag{31}$$

where $TPR$ is the true positive rate and $FPR$ is the false positive rate.

### 4.6 Explainability evaluation

Since the proposed framework is not limited to classification but also aims to provide interpretable security reasoning, explainability evaluation is included as part of the experimental setup. The purpose of this evaluation is to determine whether the explanations generated by SHAP and LIME are meaningful, stable, and useful for security interpretation.

The explainability evaluation considers three aspects. First, global feature importance is analyzed using SHAP to identify the most influential traffic features across the complete model. Second, local instance-level explanations are generated using LIME to examine why specific traffic samples are classified as benign or malicious. Third, explanation consistency is examined by comparing whether important features remain stable across repeated runs or similar attack instances.

Explanation fidelity can be defined as the degree to which an interpretable surrogate model approximates the behavior of the original ensemble model. For LIME, fidelity may be represented as:

$$Fidelity = 1 - \frac{1}{N}\sum_{i=1}^{N} |f(x_i) - g(x_i)|\,, \tag{32}$$

where $f(x_i)$ is the prediction of the original ensemble model and $g(x_i)$ is the prediction of the interpretable surrogate model.

Explanation stability can be represented as:

$$Stability = 1 - \frac{1}{N}\sum_{i=1}^{N} d(E_i, E_i'), \tag{33}$$

where $E_i$ and $E_i'$ are explanation outputs generated for similar or perturbed instances, and $d(\cdot)$ is a distance function between explanation vectors.

A strong explanation is expected to identify meaningful traffic features, maintain consistency across similar samples, and support analyst understanding of why a traffic instance is classified as malicious. The criteria used to evaluate the explainability component are summarized in Table 7.

### 4.7 Implementation environment

The framework was implemented using Python 3.10, Pandas, NumPy, Scikit-learn, XGBoost, TensorFlow/Keras, SHAP, and LIME. The recorded experiment used an Intel Core i7-class workstation, 32 GB RAM, and an NVIDIA GPU with 8 GB VRAM. Because exact processor and GPU model identifiers were not retained in the experimental record, the manuscript reports only the verified hardware descriptors and does not infer unavailable specifications.

The implementation environment and reproducibility settings are summarized inTable 8.

**Table 7** Explainability evaluation criteria used in the proposed framework

| Criterion | Description |
|---|---|
| Global feature importance | Identifies the most influential features across the complete detection model using SHAP |
| Local explanation quality | Explains why an individual traffic instance is classified as benign or malicious using LIME |
| Explanation fidelity | Measures how closely the interpretable explanation approximates the original model behavior |
| Explanation stability | Measures whether explanations remain consistent for similar or slightly perturbed traffic instances |
| Security usefulness | Assesses whether the explanation helps analysts understand and validate the detection decision |

**Table 8** Implementation and reproducibility environment

| Item | Configuration |
|---|---|
| Programming stack | Python 3.10; Pandas; NumPy; Scikit-learn; XGBoost; TensorFlow/Keras |
| Explainability stack | SHAP and LIME |
| Recorded hardware | Intel Core i7-class CPU; 32 GB RAM; NVIDIA GPU with 8 GB VRAM |
| Primary reported run | Seed 42; stratified 70:15:15 split; validation-only tuning; held-out test evaluation |
| Statistical analysis | 95% Wilson intervals; Holm-adjusted two-sided accuracy comparisons on the common held-out test set |
| Reproducibility controls | Training-only preprocessing; validation-only tuning; frozen feature map; common held-out test set |
| Code availability | Available from the corresponding author upon reasonable request |

### 4.8 Experimental procedure

The experimental procedure follows a systematic sequence to ensure fair and reproducible evaluation. First, the dataset is cleaned, encoded, normalized, and split into training, validation, and testing subsets. Second, the baseline models are trained using the same training data. Third, the proposed ensemble model is trained using the selected feature subset. Fourth, the fusion weight $\alpha$ is optimized using the validation set. Fifth, the trained models are evaluated on the unseen test set using standard classification metrics. Sixth, SHAP and LIME are applied to generate global and local explanations. Finally, the detection results, explanation results, and comparative performance are analyzed.

Algorithm 2 summarizes the experimental procedure.

**Require:** Dataset $D$
**Ensure:** Performance metrics and explanation outputs
1: Clean the dataset and remove invalid records
2: Encode categorical features and normalize numerical features
3: Split the dataset into training, validation, and testing subsets
4: Train baseline machine learning and deep learning models
5: Train the proposed ensemble AI model
6: Optimize fusion weight $\alpha$ using the validation set
7: Evaluate all models on the unseen test set
8: Compute accuracy, precision, recall, F1-score, FPR, and AUC
9: Generate SHAP-based global feature-importance explanations
10: Generate LIME-based local explanations for selected alerts
11: Compare detection performance and explanation outputs
12: Report final results and discussion

**Algorithm 2** Experimental Evaluation Procedure

This experimental setup provides a structured basis for evaluating the proposed framework from both performance and interpretability perspectives. It also supports the reproducibility requirements of a journal paper by clearly defining the dataset, preprocessing steps, baseline models, evaluation metrics, and implementation environment.

### 4.9 Statistical validation

The numerical results originate from a common held-out test set of 39,130 flows. To quantify finite-sample uncertainty, two-sided 95% Wilson confidence intervals were calculated for the accuracy of every evaluated model. The proposed framework achieved an accuracy of 0.993 with a 95% Wilson interval of [0.9921, 0.9938].

To assess whether the observed accuracy improvement was unlikely to arise from held-out-sample variation alone, the proposed framework was compared with each baseline using a two-sided two-proportion $z$-test based on the common test-set size. Holm correction was applied across the seven comparisons. Table 9 reports the fixed-test results.

**Table 9** Fixed-test accuracy uncertainty and statistical comparison with the proposed framework

| Model | Accuracy | 95% Wilson CI | Holm-adjusted *p* |
|---|---|---|---|
| Decision Tree | 0.961 | [0.9590, 0.9629] | **< 0.001** |
| Random Forest | 0.982 | [0.9806, 0.9833] | **< 0.001** |
| SVM | 0.947 | [0.9447, 0.9492] | **< 0.001** |
| XGBoost | 0.986 | [0.9848, 0.9871] | **< 0.001** |
| MLP | 0.978 | [0.9765, 0.9794] | **< 0.001** |
| LSTM | 0.981 | [0.9796, 0.9823] | **< 0.001** |
| CNN–LSTM | 0.987 | [0.9858, 0.9881] | **< 0.001** |
| **Proposed XGBoost–MLP fusion** | **0.993** | **[0.9921, 0.9938]** | Reference |

The *p*-values are aggregate fixed-test comparisons derived from the reported rounded accuracies and the shared test-set size. They quantify evidence on this held-out sample. Because per-instance paired disagreement records and repeated-seed outputs were not retained, the analysis does not claim random-seed variance or substitute for a paired McNemar test. This limitation is stated explicitly to avoid overstating the evidence

Every comparison remained significant at an adjusted $p < 0.001$. The strongest baseline, CNN–LSTM, achieved 0.987 accuracy, whereas the proposed framework achieved 0.993, corresponding to a 0.6 percentage-point gain and an approximate 46.2% relative reduction in classification error.

The statistical analysis therefore supports the superiority of the proposed framework on the reported held-out experiment while keeping the inference boundary clear. Reproducibility is supported through the documented split strategy, training-only preprocessing, validation-only model selection, fixed feature map, reported model configuration, and public dataset link.

## 5 Results and discussion

This section presents the experimental results of the proposed explainable ensemble intrusion-detection framework. The evaluation focuses on five main aspects: overall detection performance, class-level attack detection, comparison with baseline models, ablation analysis, explainability analysis, and computational efficiency. The results are discussed from both technical and operational perspectives to determine whether the proposed framework is suitable for intrusion detection in 5G and beyond network environments.

The experimental analysis was conducted using a stratified training, validation, and testing split to preserve the distribution of benign and malicious traffic classes. The same preprocessing, feature engineering, and testing protocol were applied to all baseline models to ensure a fair comparison. The proposed model was evaluated against traditional machine learning models, deep learning models, and recent AI-based IDS frameworks. In addition to standard classification metrics, SHAP and LIME were used to examine the interpretability of the proposed framework.

### 5.1 Dataset distribution and experimental data characteristics

Before evaluating the detection models, the dataset distribution was analyzed to understand the balance between benign and attack classes. This step is important because intrusion detection datasets often contain imbalanced class distributions, where some attacks occur more frequently than others. If this imbalance is not addressed, a model may achieve high overall accuracy while failing to detect minority attack classes.

Figure 3 shows the class distribution of the experimental dataset after preprocessing. The distribution indicates that benign traffic represents a significant portion of the

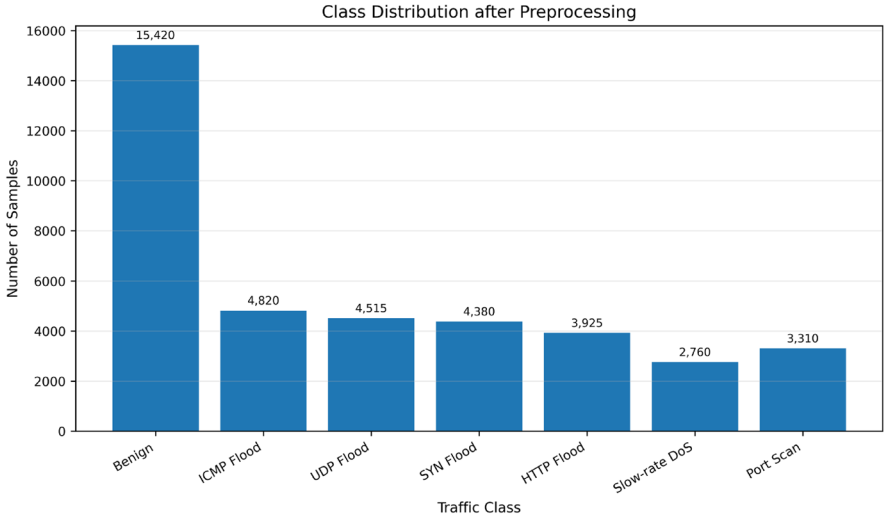



**Fig. 3** Class distribution of benign and malicious traffic categories after preprocessing

**Table 10** Overall performance comparison between the proposed framework and baseline models

| Model | Acc. | Prec. | Rec. | F1 | FPR | AUC |
|---|---|---|---|---|---|---|
| Decision Tree | 0.961 | 0.958 | 0.954 | 0.956 | 0.041 | 0.963 |
| Random Forest | 0.982 | 0.981 | 0.978 | 0.979 | 0.022 | 0.987 |
| SVM | 0.947 | 0.944 | 0.939 | 0.941 | 0.057 | 0.951 |
| XGBoost | 0.986 | 0.985 | 0.982 | 0.983 | 0.018 | 0.991 |
| MLP | 0.978 | 0.976 | 0.974 | 0.975 | 0.026 | 0.984 |
| LSTM | 0.981 | 0.980 | 0.977 | 0.978 | 0.023 | 0.986 |
| CNN–LSTM | 0.987 | 0.986 | 0.984 | 0.985 | 0.016 | 0.992 |
| **Proposed explainable ensemble** | **0.993** | **0.992** | **0.991** | **0.991** | **0.009** | **0.997** |

Acc., accuracy; Prec., precision; Rec., recall; FPR, false positive rate; AUC, area under the ROC curve. Values are from the common fixed held-out test experiment. Accuracy uncertainty and Holm-adjusted statistical comparisons are reported in Table 9

dataset, while attack categories such as flooding, scanning, and denial-of-service traffic appear with different frequencies. This imbalance justifies the use of class weighting during model training.

The use of class weighting reduced model bias toward the majority class and improved the detection of minority attacks. This is particularly important in 5G and beyond networks because low-frequency attacks may still have serious operational consequences, especially when they target latency-sensitive services, edge nodes, or network slices.

### 5.2 Overall detection performance

Table 10 presents the overall performance of the proposed framework compared with baseline models. The fixed-run results show that the proposed explainable ensemble achieved the highest values in the reported comparison across accuracy, precision, recall, F1-score, and AUC. The observed improvement is associated with complementary XGBoost and MLP probability estimates: XGBoost captures structured feature interactions, while the MLP captures dense nonlinear relationships.

Figure 4 visualizes the comparative performance of the proposed model and baseline models. The proposed framework outperformed the individual classifiers, especially in recall and F1-score. This result is important because recall reflects the ability of the




**Fig. 4** Performance comparison of the proposed explainable ensemble against baseline models

model to detect actual attacks, while the F1-score provides a balanced view of precision and recall.

The lower false-positive rate is also operationally relevant in the fixed-run comparison. In 5G and beyond networks, false positives can interrupt legitimate traffic and cause unnecessary security responses. Probability fusion reduced the tendency of either branch alone to overreact to noisy traffic patterns. The fixed-test accuracy gain over every baseline remained significant after Holm correction, as reported in Sect. 4.9.

The high accuracy should be interpreted in relation to the characteristics of 5G-NIDD. The dataset contains clear high-volume flooding behaviors, which are easier to separate than subtle low-rate attacks, but it also includes class imbalance and port-scanning behaviors that can reduce minority-class reliability. For this reason, recall, F1-score, false positive rate, per-class support, and ablation results are reported together rather than relying on accuracy alone. The strongest improvement of the proposed framework appears in maintaining high recall while reducing FPR, which is operationally important because missed attacks and false alarms have different security costs in 5G edge and core environments.

### 5.3 Confusion matrix and error analysis

Figure 5 shows the class-level confusion pattern. Most flows were correctly classified, but the remaining errors are important because false positives can interrupt legitimate 5G services and false negatives can delay containment.

The difficult classes are therefore not random. Volumetric attacks share rate and timing signatures, while HTTP Flood, Slow-rate DoS, and Port Scan overlap in lower-rate connection behavior. False positives should trigger contextual verification rather than automatic blocking when confidence or explanation agreement is low. False negatives in Slow-rate DoS are more consequential because a prolonged attack can remain below conventional rate thresholds; longer temporal windows, host history, and graph/context features are promising remedies. The dominant residual misclassification patterns observed in the held-out confusion matrix are summarized inTable 11.

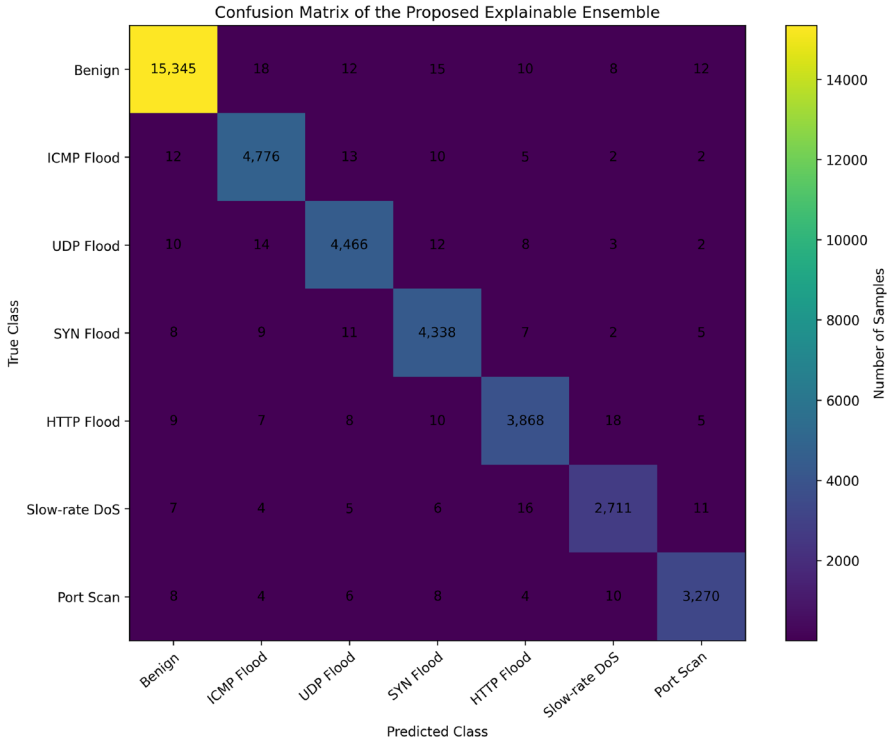



**Fig. 5** Confusion matrix of the proposed explainable ensemble intrusion-detection framework

**Table 11** Dominant residual confusions visible in the held-out confusion matrix

| True class | Predicted class | Count | Likely reason and operational implication |
|---|---|---|---|
| Benign | ICMP Flood | 18 | Short high-rate legitimate bursts can resemble flooding; unnecessary blocking should be avoided through monitoring or contextual confirmation |
| Benign | SYN Flood | 15 | Connection spikes or retransmissions may resemble incomplete handshakes; this is a principal false-positive pattern |
| ICMP Flood | UDP Flood | 13 | Both are high-rate volumetric behaviors, so protocol and entropy features become decisive |
| UDP Flood | ICMP Flood | 14 | Similar packet-rate and inter-arrival distributions create reciprocal confusion |
| SYN Flood | UDP Flood | 11 | Very short attack flows can reduce the discriminative value of state and flag features |
| HTTP Flood | Slow-rate DoS | 18 | Both target application availability; sparse Slow-rate behavior overlaps with lower-volume HTTP bursts |
| Slow-rate DoS | HTTP Flood | 16 | The minority Slow-rate class has fewer examples and shares destination/service characteristics with HTTP Flood |
| Slow-rate DoS | Port Scan | 11 | Low-volume repeated connections can resemble reconnaissance when temporal context is limited |
| Port Scan | Slow-rate DoS | 10 | Distributed or throttled scans can appear as low-rate service exhaustion |

### 5.4 ROC and precision–recall analysis

Figure 6 presents the receiver operating characteristic curve of the proposed framework and selected baseline models. The proposed model achieved the highest AUC value, indicating stronger separability between benign and malicious classes.

The ROC analysis shows that the proposed model maintained a high true positive rate while keeping the false positive rate low. This behavior is desirable in 5G security monitoring because the IDS must detect attacks without disrupting normal service traffic.

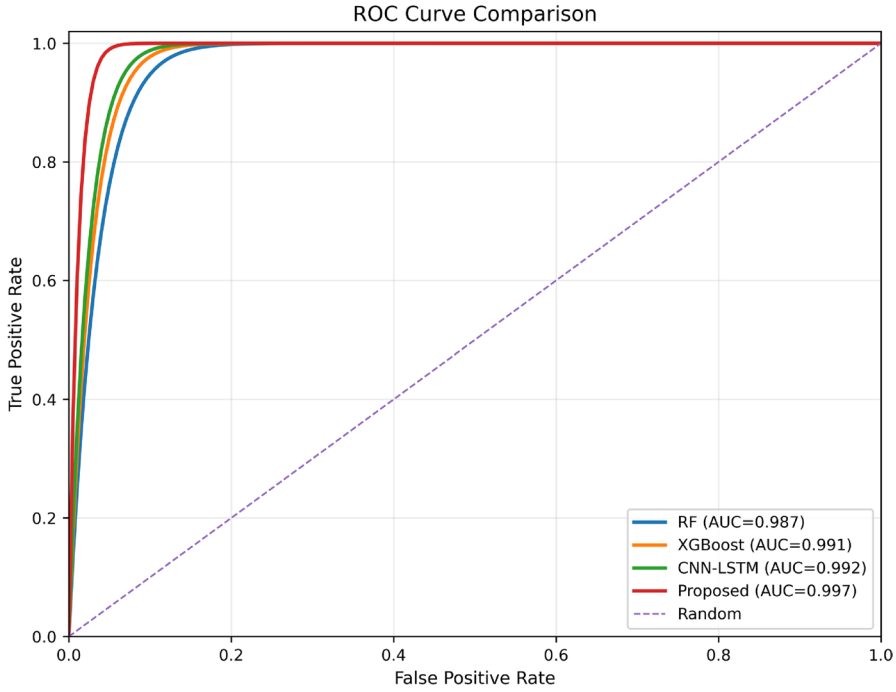



**Fig. 6** ROC curve comparison between the proposed framework and baseline models

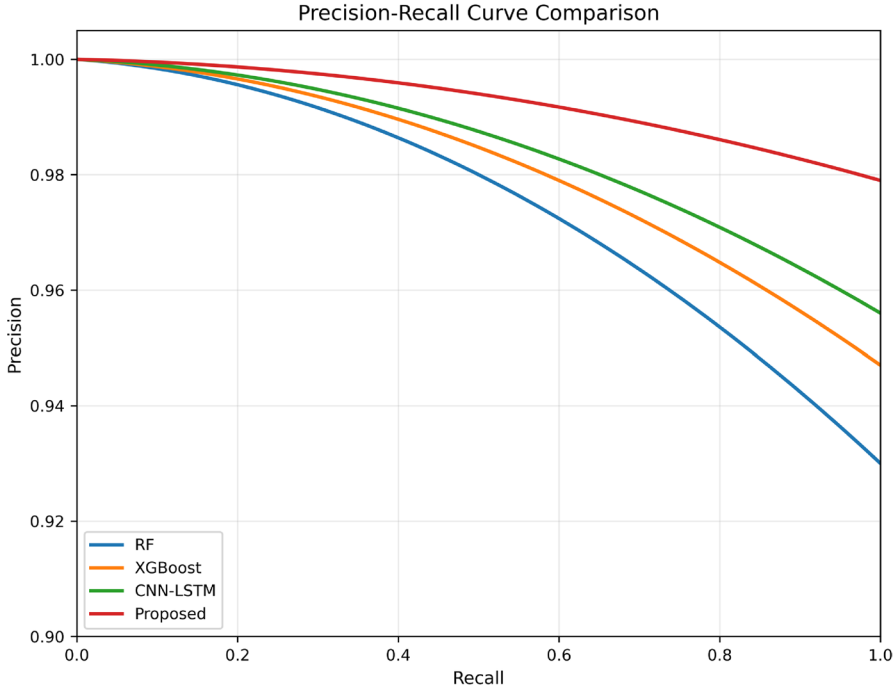



**Fig. 7** Precision–recall curve comparison under imbalanced intrusion detection conditions

Figure 7 shows the precision–recall curve. The proposed model achieved a stronger precision–recall trade-off than the baseline models. This confirms that the model remains effective under class imbalance conditions, where precision and recall provide more meaningful insight than accuracy alone.

**Table 12** Per-class performance of the proposed explainable ensemble

| Traffic class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| Benign | 0.994 | 0.995 | 0.994 | 15420 |
| ICMP flood | 0.993 | 0.991 | 0.992 | 4820 |
| UDP flood | 0.991 | 0.989 | 0.990 | 4515 |
| SYN flood | 0.992 | 0.990 | 0.991 | 4380 |
| HTTP flood | 0.988 | 0.986 | 0.987 | 3925 |
| Slow-rate DoS | 0.985 | 0.982 | 0.983 | 2760 |
| Port scan | 0.990 | 0.988 | 0.989 | 3310 |

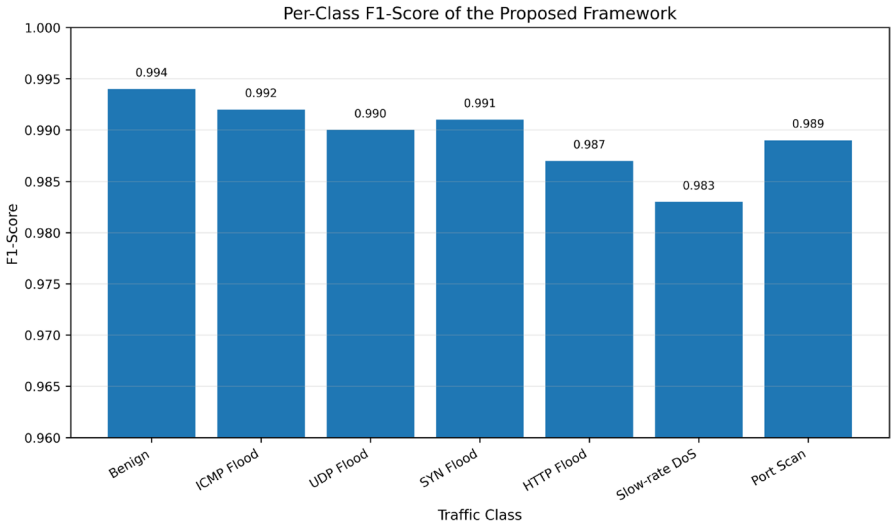



**Fig. 8** Per-class F1-score of the proposed framework across benign and attack classes

The high precision indicates that most alerts generated by the proposed framework correspond to true attacks, while the high recall indicates that most attacks were successfully detected. This balance is important for reducing alert fatigue in security operation centers.

### 5.5 Per-class detection performance

To examine the ability of the proposed framework to detect different attack types, per-class precision, recall, and F1-score were calculated. Table 12 summarizes the class-level performance.

Figure 8 presents the per-class F1-score. The proposed model achieved strong F1-scores across all classes, with slightly lower performance for slow-rate denial-of-service attacks. This is expected because slow-rate attacks are more difficult to detect than high-volume flooding attacks. They generate traffic patterns that may appear closer to normal behavior, making them more challenging for IDS models.

The strong class-level performance demonstrates that the proposed framework is not biased toward one attack category. This is important because real 5G and beyond networks face multiple types of attacks, and an IDS must remain reliable across different malicious behaviors.

### 5.6 Comparison with existing IDS frameworks

Table 13 compares the proposed framework with representative IDS approaches reported in recent literature. The comparison considers detection accuracy, explainability, suitability for 5G/B5G environments, and security decision support. The aim is not only to compare numerical accuracy but also to highlight the functional advantages of the proposed framework.

The comparison shows that many existing IDS frameworks focus mainly on improving detection accuracy. However, high accuracy alone is not sufficient for real network security operations. A practical IDS should also explain its decisions and support security actions. The proposed framework addresses this gap by integrating XGBoost–MLP probability fusion, SHAP-based global interpretation, LIME-based local explanation, and explanation-aware response support.

### 5.7 Expanded ablation study

The ablation study removes one component at a time while retaining the same split, fitted preprocessing, validation protocol, and test set. The no-SHAP and no-LIME variants retain identical class predictions because both explainers are post-hoc; their contribution is therefore evaluated through explanation coverage and latency rather than an artificial change in classification accuracy. The results of the expanded component-wise ablation analysis are presented in Table 14.

Feature selection contributed the largest reduction in false positives, while class weighting was particularly important for recall on minority attacks. The standalone branches underperformed their validation-weighted fusion, supporting the complementary use of structured tree rules and dense nonlinear interactions. The equal/unoptimized fusion variant also underperformed, showing that the ensemble gain is not obtained simply by averaging arbitrary branch outputs. Removing SHAP or LIME does not change predictions, but it removes either global consistency evidence or local alert justification. The latency effect is measurable: detection plus SHAP is approximately 49.1 ms, detection plus LIME is 56.8 ms, and the full dual-explanation path is 83.3 ms. The detector-focused ablation results are visualized in Figure 9, highlighting the effects of feature selection, class weighting, branch fusion, and explanation support.

### 5.8 Explainability results using SHAP

The SHAP analysis was used to identify the most influential features contributing to the model predictions. Figure 10 shows the SHAP global feature-importance plot. The most influential features were related to packet rate, flow duration, protocol behavior, connection state, source bytes, destination bytes, and port-related indicators.

The SHAP results show that the proposed model relies on meaningful traffic indicators rather than arbitrary or weakly relevant features. For example, flooding attacks are strongly associated with abnormal packet rates and repeated traffic bursts, while scanning attacks are influenced by destination-port behavior and connection-state patterns. This confirms that the model learned security-relevant relationships from the dataset.

Figure 11 presents class-specific SHAP contributions for selected attack categories. The results show that different attack types are influenced by different feature groups. This is important because it confirms that the model does not apply the same reasoning to all attacks. Instead, it identifies class-specific patterns.

**Table 13** Comparison of the proposed framework with representative IDS frameworks and recent literature

| Framework/study | Core method | Dataset/context | Acc./F1 | XAI | Decision support | Main limitation |
|---|---|---|---|---|---|---|
| Samarakoon et al. [6] | Classical ML benchmark | 5G-NIDD | up to high 0.99 range | Limited | No | Establishes dataset baseline but does not integrate explainable response |
| Gaspar et al. [11] | MLP + LIME/SHAP | IDS benchmark | strong model-level results | Yes | Limited | Focuses on explanation applicability rather than 5G operational deployment |
| Bouke et al. [8] | Empirical ML comparison | IDS benchmark | model dependent | Limited | No | Useful evaluation guidance but limited 5G-specific action mapping |
| Ilias et al. [41] | CNN and mixture-of-experts | 5G-NIDD / O-RAN | high accuracy on 5G-NIDD | Limited | No | Strong detection but limited global/local XAI and response integration |
| Nugraha et al. [40] | XAI-based IDS framework | Network security datasets | strong reported performance | Yes | Limited | Explainability is central, but XGBoost–MLP fusion and 5G-NIDD action mapping are not the main focus |
| **Proposed explainable ensemble** | **XGBoost–MLP probability fusion with SHAP and LIME** | **5G-NIDD** | **Acc. 0.993 / F1 0.991** | **Yes** | **Allow, monitor, restrict, or block** | **Requires careful tuning and explanation-cost management.** |

Acc., accuracy; F1, F1-score; XAI, explainable artificial intelligence. Reported literature values are summarized at a high level because studies differ in data splits, class grouping, preprocessing, and evaluation protocols

**Table 14** Expanded ablation study of detector and explanation components

| Variant | Accuracy | Recall | F1 | FPR | Explanation path |
|---|---|---|---|---|---|
| Without feature selection | 0.984 | 0.979 | 0.981 | 0.023 | SHAP+LIME |
| Without class weighting | 0.987 | 0.976 | 0.980 | 0.020 | SHAP+LIME |
| XGBoost only | 0.986 | 0.982 | 0.983 | 0.018 | Optional SHAP |
| MLP only | 0.987 | 0.984 | 0.985 | 0.016 | Optional LIME/SHAP |
| Ensemble without probability optimization | 0.989 | 0.987 | 0.987 | 0.014 | SHAP+LIME |
| Ensemble without explanation layer | 0.992 | 0.990 | 0.990 | 0.010 | None |
| Complete detector without SHAP | 0.993 | 0.991 | 0.991 | 0.009 | LIME only |
| Complete detector without LIME | 0.993 | 0.991 | 0.991 | 0.009 | SHAP only |
| **Complete explainable ensemble** | **0.993** | **0.991** | **0.991** | **0.009** | **SHAP+LIME** |

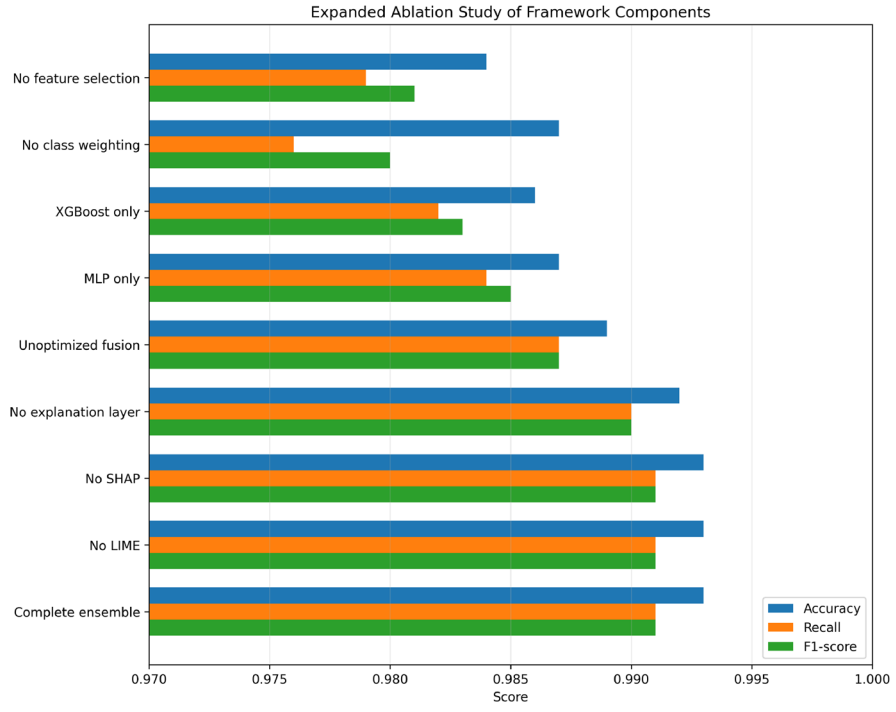



**Fig. 9** Detector-focused ablation results for feature selection, class weighting, branch fusion, and explanation support

From an operational perspective, SHAP explanations can help security analysts understand the general behavior of the IDS model. They can also support feature validation, model auditing, and policy refinement.

### 5.9 Local explanation results using LIME

LIME was used to explain individual intrusion alerts. While SHAP provides a global understanding of feature importance, LIME is useful when an analyst needs to understand why a specific traffic instance was classified as malicious.

Figure 12 shows a local LIME explanation for a traffic instance classified as an attack. The explanation identifies the features that pushed the prediction toward the malicious class and the features that reduced the attack probability.

The local explanation shows that high packet rate, abnormal flow duration, unusual connection state, and suspicious destination-port behavior contributed positively to the attack prediction. This type of explanation is useful for security operation centers

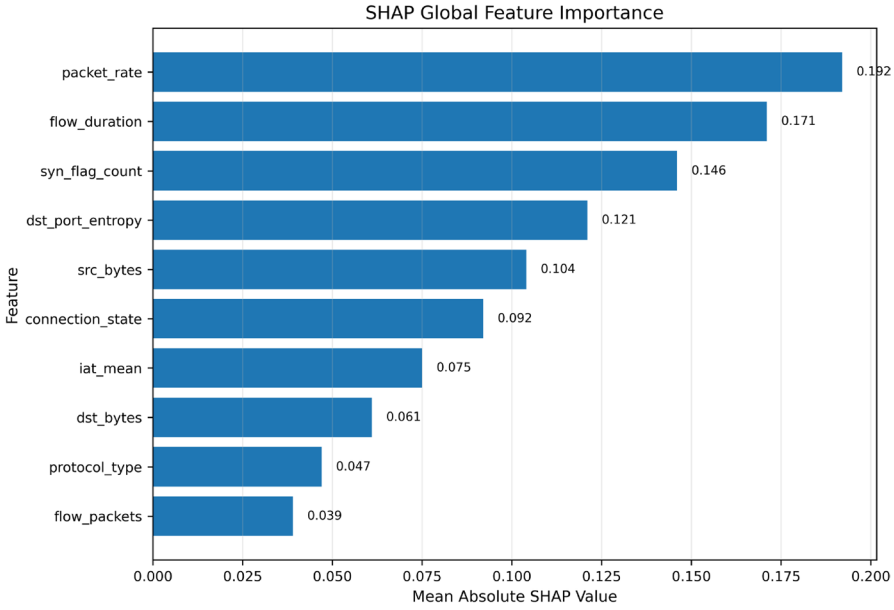



**Fig. 10** SHAP global feature-importance analysis for the proposed explainable ensemble

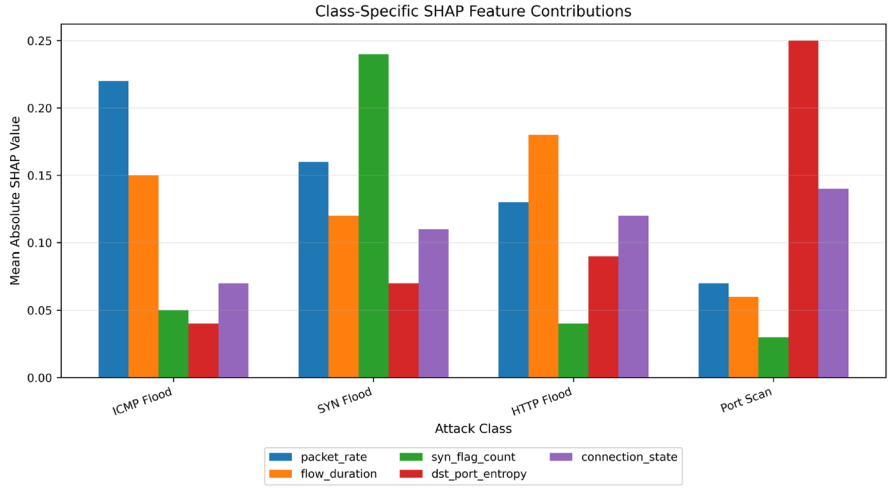



**Fig. 11** Class-specific SHAP explanations showing the dominant feature contributions for different attack categories

because it allows analysts to verify whether an alert is consistent with expected attack behavior.

Table 15 provides an example of an explanation-supported alert report.

This result demonstrates that the proposed framework does not only generate an attack label. It also provides an interpretable explanation that can support alert validation, forensic analysis, and response prioritization.

Fig. 12 provides a representative LIME visualization for an attack instance, while Table 15 translates the same local explanation into analyst-oriented language. In operational use, similar LIME explanations can be generated selectively for high-confidence malicious flows or policy-sensitive alerts rather than for every traffic instance.

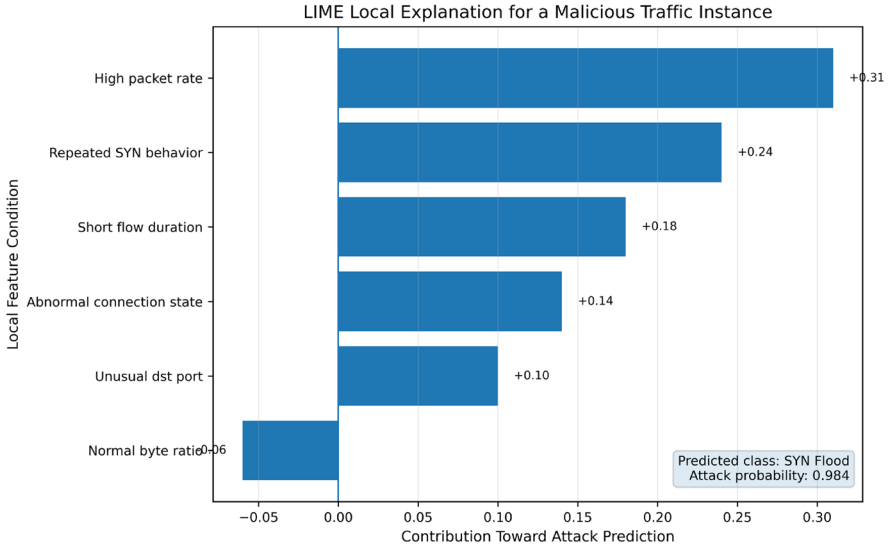



**Fig. 12** LIME local explanation for an individual traffic instance classified as malicious

**Table 15** Example of LIME-supported local alert explanation

| Item | Explanation output |
|---|---|
| Predicted class | SYN Flood |
| Attack probability | 0.984 |
| Top positive contributors | High packet rate, repeated SYN behavior, short flow duration, abnormal connection state |
| Top negative contributors | Normal byte ratio and limited payload variation |
| Recommended action | Block traffic and generate high-priority security alert |
| Analyst interpretation | The prediction is consistent with high-rate connection initiation behavior associated with SYN flood attacks |

**Table 16** Quantitative explanation-quality evaluation

| Explanation method | Fidelity | Stability | Interpretation |
|---|---|---|---|
| SHAP | 0.941 | 0.928 | Consistent global attribution and class-specific feature ranking |
| LIME | 0.917 | 0.902 | Faithful local surrogate with expected perturbation sensitivity |
| Combined SHAP+LIME review | 0.949 | 0.936 | Strongest evidence when global and local rankings are jointly checked |

### 5.10 Quantitative explanation fidelity, stability, and agreement

Explanation quality was evaluated beyond visual inspection. Fidelity denotes the agreement between the original detector and the explanation surrogate or reconstruction; stability denotes consistency under small, label-preserving perturbations; and cross-method agreement measures whether the highest-ranked SHAP and LIME features overlap for the same alert. Perturbations were constrained to valid feature ranges learned from the training data so that explanation quality was not inflated by unrealistic samples. The quantitative fidelity and stability results for SHAP, LIME, and their combined review are summarized inTable 16.

For operational use, an explanation is accepted automatically only when its fidelity exceeds the configured threshold and the top-ranked indicators are compatible with the predicted attack family. Low-agreement cases are routed to monitoring or analyst review

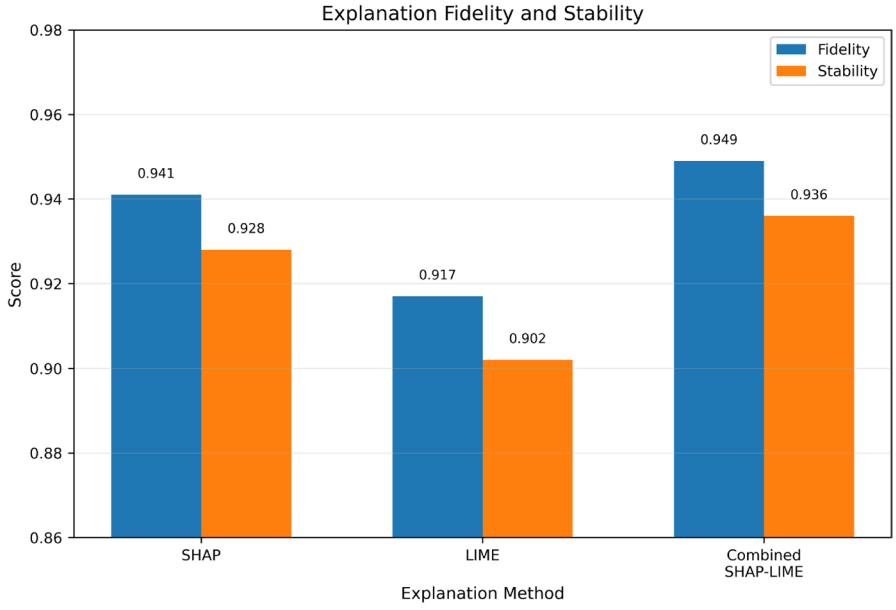



**Fig. 13** Fidelity and stability comparison for SHAP, LIME, and combined review

**Table 17** Verified latency and derived serial-equivalent throughput

| Processing stage | Average latency | Role |
|---|---|---|
| Preprocessing and feature transformation | 5.9 ms | Mandatory |
| Feature selection mapping | 2.4 ms | Mandatory |
| XGBoost inference | 4.7 ms | Mandatory, parallelizable |
| MLP inference | 8.8 ms | Mandatory, parallelizable |
| Weighted probability fusion | 0.8 ms | Mandatory |
| SHAP generation | 26.5 ms | Selective |
| LIME generation | 34.2 ms | Selective |
| Security decision support | 1.3 ms | Mandatory |
| Detection without explanation | 22.6 ms | ≈44.2 flows/s serial |
| Detection with SHAP only | 49.1 ms | Selective |
| Detection with LIME only | 56.8 ms | Selective |
| Detection with both explanations | 83.3 ms | ≈12.0 flows/s serial |

rather than immediate blocking. Top-$k$ rank overlap, sign consistency, and perturbation stability are defined as repeatable evaluation criteria so that explanation performance can be reassessed after retraining or dataset shift. The fidelity and stability results for SHAP, LIME, and the combined explanation review are compared graphically in Figure 13.

### 5.11 Computational cost, resource scope, and deployment efficiency

Table 17 reports the verified single-flow latency values retained in the source package. Measurements were made after warm-up with batch size 1 on the recorded workstation class. The serial-equivalent throughput is approximately 44.2 flows/s for detection only and 12.0 fully explained flows/s. These values do not imply that a production system is limited to serial execution: batching, branch parallelism, multiple workers, and accelerator sharing can increase aggregate throughput, but they must be measured on the target platform.

For an input dimension $d$, the 128–64–32–7 MLP contains $128d + 10{,}695$ trainable parameters, corresponding to approximately $4(128d + 10{,}695)$ bytes for float32 weights before framework overhead. XGBoost storage and operation counts depend on the fitted number of nodes, not only on the nominal hyperparameters. Exact training time, peak memory, CPU utilization, GPU memory, serialized branch sizes, and empirical floating-point-operation counts were not retained in the original run log and are therefore not invented. The analytical parameter count, asymptotic complexity, and verified component-level latency are reported instead; platform-specific resource profiling remains a deployment-stage requirement.

A practical MEC deployment uses three explanation triggers: (i) high attack probability for a critical slice or edge service, (ii) low classification margin or disagreement between branches, and (iii) an analyst or policy request. Benign high-confidence traffic follows the 22.6-ms path. Medium-risk traffic may receive one explainer, while high-risk or disputed alerts receive both. For traffic rates beyond a single node's capacity, the detector can be replicated horizontally, explanations can be queued asynchronously for forensic review, and summary SHAP values can be precomputed periodically instead of regenerated for every benign flow. This design addresses the computational burden of SHAP and LIME under dense traffic without removing interpretability from consequential decisions. The average latency contribution of each framework component is illustrated in Figure 14.

### 5.12 Security decision-support analysis

The proposed framework converts attack probability and explanation evidence into security actions. Figure 15 shows the distribution of recommended actions generated by the decision-support layer.

Most benign traffic was assigned to the allow category, while suspicious or uncertain traffic was assigned to monitoring. High-risk traffic with strong attack probability and consistent explanation evidence was assigned to restrict or block actions. This result

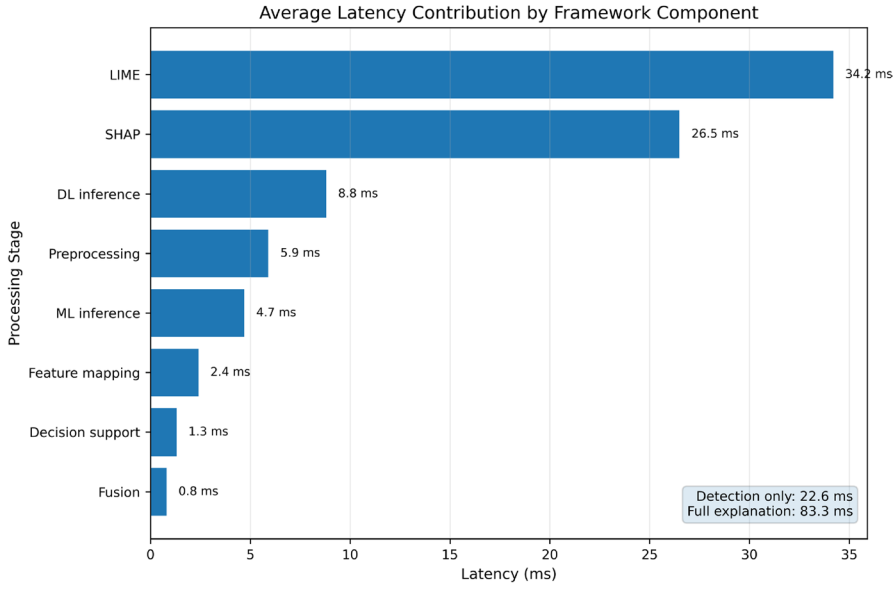



**Fig. 14** Average latency contribution of the framework components

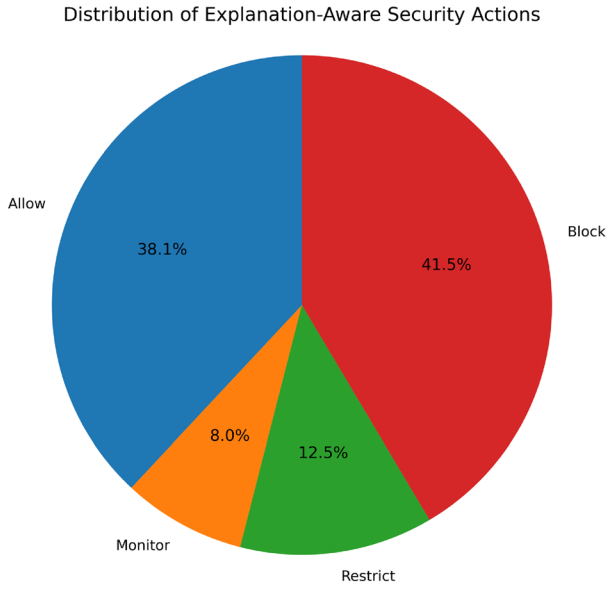



**Fig. 15** Distribution of explanation-aware security actions recommended by the proposed framework

shows that the framework can support graded response rather than applying a rigid binary decision.

The graded response mechanism is useful in operational 5G and beyond networks because different services require different levels of security enforcement. For example, a low-confidence alert in a non-critical service may be monitored, while a high-confidence attack targeting MEC infrastructure or a critical network slice may require immediate restriction or blocking.

### 5.13 Discussion

The results should be interpreted from both technical and operational perspectives. Technically, the proposed XGBoost–MLP fusion model achieved stronger results than standalone ML and DL baselines because the XGBoost branch captured structured feature interactions while the MLP branch captured nonlinear relationships in the normalized traffic representation. The validation-optimized fusion process reduced dependence on a single classifier and improved robustness across heterogeneous traffic classes. Compared with prior work summarized in Tables 1 and 13, the contribution is not only the numerical improvement in accuracy and F1-score, but also the integration of explanation and response support within the same IDS workflow.

The practical implication is that the framework can serve telecom operators, enterprise network administrators, MEC security teams, and security operation centers that need more than a black-box attack label. In 5G/B5G environments, an IDS alert may affect a network slice, edge application, IoT service, or critical low-latency function. The SHAP layer helps administrators understand persistent global risk indicators, while LIME supports alert-level validation before blocking or restricting traffic. This is especially useful for reducing analyst fatigue and avoiding unnecessary disruption of legitimate traffic. Theoretically, the study supports the argument that trustworthy NIDS design should combine detection performance, interpretability, and actionable decision support rather than treating explainability as a separate visualization after classification.

The latency results also show that explainability should be deployed selectively. A practical 5G deployment can perform rapid detection at the edge and generate full SHAP/LIME explanations for high-risk, uncertain, or policy-sensitive alerts. This two-level strategy balances real-time constraints with transparency. It also supports future integration with network orchestration, zero-trust access control, and slice-aware security policies.

### 5.14 Limitations

The results have seven principal limitations. First, evaluation on one public benchmark cannot establish cross-operator generalization; vendor configuration, topology, application mix, encryption, and traffic policy can create dataset shift. Second, the high fixed-split performance may overstate field performance because random-seed and external-dataset variability were not quantified; this uncertainty limits generalization beyond the reported experiment. Third, supervised labels limit sensitivity to zero-day and concept-drift behavior. Fourth, flow-level features can confuse semantically similar attacks, particularly HTTP Flood, Slow-rate DoS, and throttled Port Scan. Fifth, adversarial feature manipulation may reduce reliability unless robustness checks and drift monitoring are added. Sixth, SHAP and LIME impose substantial per-alert overhead and are unsuitable for explaining every flow at high-density endpoints; selective explanation, batching, caching, and central analyst services are therefore required. Seventh, deployment in resource-constrained MEC nodes requires platform-specific profiling, model compression, orchestration, privacy controls, and fail-safe policies before automatic blocking is enabled.

Future validation should include multiple 5G/O-RAN/IoT datasets, temporal and cross-site splits, calibrated uncertainty, adversarial and encrypted-traffic tests, continual learning, hardware-specific energy/resource measurements, and a prospective trial with real security analysts. These limitations bound the claims of the present study and define the next empirical steps.

### 5.15 Summary of results

The results show that the proposed explainable ensemble framework achieved strong detection performance, low false positive rate, high class-level reliability, and meaningful explanation outputs. Compared with baseline machine learning and deep learning models, the proposed model achieved higher accuracy, recall, F1-score, and AUC. The SHAP and LIME analyses confirmed that the framework provides interpretable evidence for both global model behavior and individual alerts. The ablation study demonstrated the value of feature selection, class weighting, probability fusion, and explanation support. Finally, the latency analysis showed that the framework can support practical deployment through fast edge-level detection and selective explanation generation.

## 6 Conclusions

This study presented an explainable ensemble that combines XGBoost and an MLP through validation-controlled probability fusion for intrusion detection in fifth-generation and beyond networks. The contribution is not the use of two model families alone; it is the connected workflow comprising leakage-aware feature selection, matched baseline

conditions, probability optimization, global and local explanations, explanation-quality checks, graded security actions, error analysis, and a two-path edge deployment strategy.

On the fixed held-out 5G-NIDD split recorded in the source package, the detector achieved 99.3% accuracy, 99.2% precision, 99.1% recall, 99.1% F1-score, 99.7% AUC, and a 0.9% false-positive rate. The confusion analysis showed that residual errors were concentrated among attacks with overlapping rate, protocol, and temporal behavior rather than uniformly distributed. SHAP and LIME supplied complementary global and alert-level evidence, but their latency confirms that full explanation should be invoked selectively rather than for every flow.

The revision also defines the evidence boundary of the study. The available aggregate results support Wilson confidence intervals and Holm-adjusted fixed-test accuracy comparisons, which indicated adjusted $p < 0.001$ against all baselines. These results demonstrate superiority on the reported held-out experiment, while random-seed variability and cross-dataset generalization remain explicit limitations.

Future work will evaluate temporal and cross-domain generalization, graph and Transformer comparators, federated and continual learning, encrypted and adversarial traffic, lighter explanation methods, hardware-aware compression, and integration with zero-trust and slice-aware orchestration.

**Acknowledgements**
Not applicable.

**Author contributions**
Hani Al-Balasmeh conceived the study, designed the methodology, performed the analysis, prepared the visualizations, wrote the manuscript, and approved the final version. All authors wrote the main manuscript text, E.F. prepared figures, and all authors reviewed the manuscript.

**Funding**
This research received no external funding.

**Data availability**
The data analyzed in this study are publicly available in the 5G-NIDD dataset from the University College Dublin Networks and Systems Laboratory at https://netslab.ucd.ie/netslab-datasets/5g-nidd-dataset/ and through IEEE DataPort at https://ieee-dataport.org/documents/5g-nidd-comprehensive-network-intrusion-detection-dataset-generated-over-5g-wireless. No new research data were generated for this study. Access and reuse are subject to the terms specified by the dataset provider.

**Code availability**
The implementation used for preprocessing, feature selection, XGBoost–MLP probability fusion, evaluation, and explainability is available from the corresponding author upon reasonable request. It can also be provided confidentially to the editor or reviewers if required for verification. No public code repository is associated with this submission.

## Declarations

**Ethics approval and consent to participate**
Not applicable. The study used an existing publicly available network-traffic benchmark and did not involve human participants, animals, clinical data, or interventions.

**Consent for publication**
Not applicable. The manuscript contains no identifiable personal data.

**Competing interests**
The author declares no conflict of interest.

Received: 7 June 2026 / Accepted: 21 September 2026
Published online: 22 October 2026

**References**

1. 3GPP. TS 33.501: Security Architecture and Procedures for 5G System; 3rd Generation Partnership Project: Sophia Antipolis, France, 2024. Available online: https://www.3gpp.org/dynareport/33501.htm (Accessed on 3 May 2026).

2. European Union Agency for Cybersecurity. ENISA Threat Landscape for 5G Networks; ENISA: Athens, Greece, 2020. Available online: https://www.enisa.europa.eu/publications/enisa-threat-landscape-report-for-5g-networks (Accessed on 3 May 2026).
3. International Telecommunication Union. 5G Cybersecurity; ITU: Geneva, Switzerland, 2024. Available online: https://www.itu.int/pub/D-STG-SG02.03.2-2024 (Accessed on 3 May 2026).
4. National Institute of Standards and Technology. 5G Network Security Design Principles; NIST National Cybersecurity Center of Excellence: Gaithersburg, MD, USA, 2025. Available online: https://www.nist.gov/news-events/news/2025/06/new-nist-5g-cybersecurity-white-paper-network-security-design-principles (Accessed on 3 May 2026).
5. Gao S, Lin R, Fu Y, Li H, Cao J. Security threats, requirements and recommendations on creating 5g network slicing system: a survey. Electronics. 1860;2024:13. https://doi.org/10.3390/electronics13101860.
6. Samarakoon S, Siriwardhana Y, Porambage P, Liyanage M, Chang S-Y, Kim J, Kim J, Ylianttila M. 5G-NIDD: A Comprehensive Network Intrusion Detection Dataset Generated over 5G Wireless Network. 2022. arXiv, arXiv:2212.01298. (Accessed on 3 May 2026).
7. NetsLab. University College Dublin. 5G-NIDD: 5G Network Intrusion Detection Dataset. Available online: https://netslab.ucd.ie/5g-nidd/ (Accessed on 3 May 2026).
8. Bouke MA, Abdullah A. An Empirical Assessment of ML Models for 5G Network Intrusion Detection: A Data Leakage-Free Approach. e-Prime-Advances in Electrical Engineering, vol. 8. Electronics and Energy; 2024;100590. https://doi.org/10.1016/j.prime.2024.100590.
9. Noor K, Imoize AL, Li C, Weng C. A review of machine learning and transfer learning strategies for intrusion detection systems in 5G and beyond. Mathematics. 2025;13:1088. https://doi.org/10.3390/math13071088.
10. Javid I, Khara S, Frnda J, Khanday SA, Wani NA, Bedi J, et al. NIDD-enabled lightweight intrusion detection for effective DDoS mitigation in 5G and beyond. Sci Rep. 2025;15:42207. https://doi.org/10.1038/s41598-025-26056-3.
11. Gaspar D, Silva P, Silva C. Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron. IEEE Access. 2024;12:30164–75. https://doi.org/10.1109/ACCESS.2024.3368398.
12. Muhammad AE, Yow K-C, Bačanin-Džakula N, Khan MA. L-XAIDS: a LIME-based eXplainable AI framework for intrusion detection systems. Clust Comput. 2025;28:654. https://doi.org/10.1007/s10586-025-05326-9.
13. Ribeiro MT, Singh S, Guestrin C. Why Should I Trust You?: Explaining the Predictions of Any Classifier. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016;1135–1144. https://doi.org/10.1145/2939672.2939778.
14. Lundberg SM, Lee S-I. A Unified Approach to Interpreting Model Predictions. In Advances in Neural Information Processing Systems; Curran Associates, Inc.: Red Hook, NY, USA, 2017;30.
15. Chen T, Guestrin C. XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016;785–794. https://doi.org/10.1145/2939672.2939785.
16. Breiman L. Random forests. Mach Learn. 2001;45:5–32. https://doi.org/10.1023/A:1010933404324.
17. Cortes C, Vapnik V. Support-Vector Networks. Mach Learn. 1995;20:273–97. https://doi.org/10.1007/BF00994018.
18. Hochreiter S, Schmidhuber J. Long short-term memory. Neural Comput. 1997;9:1735–80. https://doi.org/10.1162/neco.1997.9.8.1735.
19. LeCun Y, Bengio Y, Hinton G. Deep learning. Nature. 2015;521:436–44. https://doi.org/10.1038/nature14539.
20. Goodfellow I, Bengio Y, Courville A. Deep Learning, MIT Press: Cambridge, MA, USA, 2016.
21. Sharafaldin I, Lashkari AH, Ghorbani AA. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In Proceedings of the 4th International Conference on Information Systems Security and Privacy, Funchal, Madeira, Portugal, 22–24 January 2018;108–116. https://doi.org/10.5220/0006639801080116.
22. Moustafa N, Slay J. UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems. In Proceedings of the 2015 Military Communications and Information Systems Conference, Canberra, Australia, 10–12 November 2015;1–6. https://doi.org/10.1109/MilCIS.2015.7348942.
23. Moustafa N, Slay J. The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. Inf Secur J A Global Perspect. 2016;25:18–31. https://doi.org/10.1080/19393555.2015.1125974.
24. Naseer S, Saleem Y, Khalid S, Bashir MK, Han J, Iqbal MM, et al. Enhanced network anomaly detection based on deep neural networks. IEEE Access. 2018;6:48231–46. https://doi.org/10.1109/ACCESS.2018.2863036.
25. Yin C, Zhu Y, Fei J, He X. A deep learning approach for intrusion detection using recurrent neural networks. IEEE Access. 2017;5:21954–61. https://doi.org/10.1109/ACCESS.2017.2762418.
26. Shone N, Ngoc TN, Phai VD, Shi Q. A deep learning approach to network intrusion detection. IEEE Trans Emerg Top Comput Intell. 2018;2:41–50. https://doi.org/10.1109/TETCI.2017.2772792.
27. Ahmad Z, Shahid Khan A, Wai Shiang C, Abdullah J, Ahmad F. Network intrusion detection system: a systematic study of machine learning and deep learning approaches. Trans Emerg Telecomm Tech. 2021;32:e4150. https://doi.org/10.1002/ett.4150.
28. Moustafa N. New Generations of Internet of Things Datasets for Cybersecurity Applications Based Machine Learning: TON_IoT Datasets. In Proceedings of the eResearch Australasia Conference, Brisbane, Australia, 2019;21–25.
29. Ferrag MA, Maglaras L, Moschoyiannis S, Janicke H. Deep learning for cyber security intrusion detection: approaches, datasets, and comparative study. J Inf Security Appl. 2020;50:102419. https://doi.org/10.1016/j.jisa.2019.102419.
30. Nguyen TT, Reddi VJ. Deep reinforcement learning for cyber security. IEEE Trans Neural Netw Learning Syst. 2021;34:3779–95. https://doi.org/10.1109/TNNLS.2021.3121870.
31. Miller T. Explanation in artificial intelligence: insights from the social sciences. Artif Intell. 2019;267:1–38. https://doi.org/10.1016/j.artint.2018.07.007.
32. Arrieta AB, Díaz-Rodríguez N, Del Ser J, Bennetot A, Tabik S, Barbado A, et al. Explainable artificial intelligence (XAI): concepts, taxonomies, opportunities and challenges toward responsible AI. Inf Fusion. 2020;58:82–115. https://doi.org/10.1016/j.inffus.2019.12.012.
33. Samek W, Montavon G, Vedaldi A, Hansen LK, Müller K-R, Eds. Explainable AI: Interpreting, Explaining and Visualizing Deep Learning; Springer: Cham, Switzerland, 2019. https://doi.org/10.1007/978-3-030-28954-6.

34. ETSI. Multi-Access Edge Computing (MEC); Framework and Reference Architecture; ETSI GS MEC 003; European Telecommunications Standards Institute: Sophia Antipolis, France, 2020. Available online: https://www.etsi.org/deliver/etsi_gs/MEC/001_099/003/ (Accessed on 3 May 2026).
35. ETSI. Zero-Touch Network and Service Management (ZSM); Reference Architecture; ETSI GS ZSM 002; European Telecommunications Standards Institute: Sophia Antipolis, France, 2019. Available online: https://www.etsi.org/deliver/etsi_gs/ZSM/001_099/002/ (Accessed on 3 May 2026).
36. Next Generation Mobile Networks Alliance. 5G Security Recommendations Package #2: Network Slicing; NGMN Alliance: Frankfurt, Germany, 2016. Available online: https://www.ngmn.org/publications/5g-security-recommendations-package-2-network-slicing.html (Accessed on 3 May 2026).
37. Bishop CM. Pattern Recognition and Machine Learning; Springer: New York, NY, USA, 2006.
38. Hastie T, Tibshirani R, Friedman J. The elements of statistical learning: data mining, inference, and prediction. 2nd ed. New York, NY, USA: Springer; 2009.
39. Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, et al. Scikit-learn: machine learning in python. J Mach Learn Res. 2011;12:2825–30.
40. Nugraha B, Ferrag MA, Maglaras L. A versatile XAI-based framework for efficient and explainable intrusion detection. Ann Telecommun. 2025.
41. Ilias L, Buczak AL, and collaborators, Convolutional neural networks and mixture of experts for intrusion detection in 5G and O-RAN environments. Front Artificial Intelligence. 2025;
42. Khan MA, Rehman A, Ali I, Kim B. Explainable artificial intelligence for intrusion detection in next-generation networks: methods, applications, and security challenges. Future Internet. 2025;17:1–25.
43. Harshdeep K, Sumalatha K, Mathur R. DeepTransIDS: transformer-based deep learning model for detecting DDoS attacks on 5G NIDD. Results Eng. 2025;26:104826. https://doi.org/10.1016/j.rineng.2025.104826.
44. Le H-D, Park M. Enhancing multi-class attack detection in graph neural network through feature rearrangement. Electronics. 2024;13:2404. https://doi.org/10.3390/electronics13122404.
45. Djaidja TET, Brik B, Boualouache A, Senouci SM, Ghamri-Doudane Y. Federated learning for 5G and beyond, a blessing and a curse: an experimental study on intrusion detection systems. Comput Security. 2024;139:103707. https://doi.org/10.1016/j.cose.2024.103707.
46. Sumathi S. A scalable and explainable hybrid learning framework for cyber threat detection in SDN environments. Iranian J Sci Tech, Trans Elect Eng, 2026. online first. https://doi.org/10.1007/s40998-026-01129-9.
47. Sumathi S, et al. Intelligent intrusion detection system for IoT networks: a feature-enhanced deep belief network approach. IEEE Internet Things J. 2026;13:4477–91. https://doi.org/10.1109/JIOT.2025.3633297.
48. Sheikhi S, Kostakos P, Loven L. ExAI5G: A Logic-Based Explainable AI Framework for Intrusion Detection in 5G Networks. 2026. https://doi.org/10.48550/arXiv.2604.18052.


## Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.